



NEW GIZA UNIVERSITY
SCHOOL OF ENGINEERING

Profit-Aware AI Framework for Credit Card Fraud Detection

Graduation Thesis

Submitted in Partial Fulfillment of the Requirements for the
Degree of
Bachelor of Engineering

Mohammad Ossama – 202201025
Ammar Ahmed – 202201291
Aly Osman – 202202499
Abdulrahman Amr – 202200373
Jana Sameh – 202201414
Mohamed Walid – 202202517

Supervised by: Dr. Mariam Nabil

Program of Computer, Communications and Autonomous
Systems

Spring 2026

Abstract

Financial Fraud detection is one of many underexplored fields in Academia, This paper demonstrates the use of Machine Learning and Reinforcement Learning for credit card financial fraud detection. This paper aims to use a Reinforcement Learning Proximal policy Optimization agent on top of a hybrid classifier that combines a denoising autoencoder with a CatBoost gradient-boosting model, in order to achieve the highest cost savings all while having low cost to develop and train. The RL agent optimizes its cost on a transaction-level trinary actions accept, reject, or review based on expected financial cost rather than classification accuracy alone. The models were trained on the IEEE-CIS Dataset using an Nvidia GeForce RTX 4050 6 GB GPU with AMD Ryzen 7-7535-HS Processor. The hybrid AE+CatBoost classifier achieved an AUC-ROC of 0.9127 on the held-out test set. This paper also explored other Machine Learning Models to be used with the RL agent including Random Forest, CNN, KDE, LOF, and standalone Autoencoder ensembles. The resulting hybrid architecture improves financial decision quality by explicitly minimizing expected fraud-related losses while maintaining high detection capability. This work demonstrates that coupling traditional machine learning with reinforcement learning offers a practical, cost-effective, and scalable solution for real-world financial fraud detection systems.

Keywords: Machine Learning- Reinforcement Learning- Random Forest- Kernel Density Estimation- Convolutional Neural Networks- Local Outlier Factor- Autoencoder- CatBoost- Deep Q-Networks -Proximal Policy Optimization -Financial Fraud Detection

Contents

1	Chapter 1: Introduction	7
1.1	Background and Motivation	7
1.2	Problem Statement	7
1.3	Project Objectives	7
1.4	Scope and Limitations	7
1.5	Significance of Study	8
1.6	Thesis Organization	8
2	Chapter 2: Literature Review	8
2.1	Introduction to Literature Review	8
2.2	Previous Research and Studies	9
2.2.1	Random Forest for Fraud Detection	10
2.2.2	Local Outlier Factor for Fraud Detection	10
2.2.3	Kernel Density Estimation for Fraud Detection	12
2.2.4	Convolutional Neural Network (CNN)	12
2.2.5	RL Cost Function	14
2.3	Current State of the Field	14
2.4	Theoretical Framework	16
2.4.1	Cost-Sensitive Learning	16
2.4.2	Anomaly Detection Theory	17
2.4.3	Reinforcement Learning for Cost Optimization	18
2.5	Research Gap	19
3	Chapter 3: Methodology	19
3.1	System Overview	19
3.1.1	Initial System Design	19
3.1.2	Final System Design	20
3.2	Data Collection and Preprocessing	21
3.3	System Design and Constraints	22
3.3.1	Autoencoder Approach	22
3.3.2	Random Forest Approach	23
3.3.3	LOF Methodology	24
3.3.4	KDE Approach	24
3.3.5	CNN Methodology	25
3.3.6	ANN Methodology	26
3.3.7	Baseline Model Evaluation and Selection	26
3.4	Evaluation Metrics	26
3.5	Cost Function Design	27
3.5.1	Initial Design	27

3.5.2	Extended Cost Function	29
3.5.3	Amount-Aware Policy	30
3.5.4	Review-Capacity Layer	32
3.5.5	Dashboard and Human-in-the-Loop Decision Support	34
4	Chapter 4: Implementation and Results	35
4.1	Implementation Details	35
4.1.1	Autoencoder Implementation	35
4.1.2	Random Forest Implementation	36
4.1.3	LOF Implementation	36
4.1.4	KDE Implementation	36
4.1.5	CNN Implementation	37
4.1.6	ANN Implementation	37
4.1.7	Reinforcement Learning Agent Implementation	38
4.1.8	Dashboard Implementation	38
4.2	Results	44
4.2.1	Autoencoder and Hybrid Classifier Results	44
4.2.2	Random Forest Results	45
4.2.3	LOF Results	45
4.2.4	KDE Results	46
4.2.5	CNN and ANN Results	46
4.2.6	Reinforcement Learning Policy Results	47
4.2.7	Capacity Layer Results	49
4.2.8	Dashboard Output and Visual Analysis	49
4.3	Analysis of Results	53
4.3.1	Autoencoder Analysis	53
4.3.2	LOF Analysis	53
4.3.3	KDE Analysis	53
4.3.4	Random Forest Analysis	53
4.3.5	CNN and ANN Analysis	54
5	Chapter 5: Discussion	54
5.1	Interpretation of Results	54
5.2	Comparison with Previous Studies	55
5.3	Limitation	55
6	Chapter 6: Conclusion and Future Work	55
6.1	Summary of Work	55
6.2	Key Contributions	55
6.3	Future Work	56

List of Figures

1	Hybrid ML-RL Fraud Detection System Architecture	19
2	Final Amount-Aware Hybrid ML-RL Fraud Detection System Architecture	20
3	Learned decision thresholds as a function of transaction amount. The review band widens with amount, directing larger transactions toward human review.	31
4	Bank Operations overview showing transaction count, blocked transactions, review workload, blocked amount, total transaction value, risk score, and review capacity indicators.	40
5	Operational decision analytics showing the action distribution and review capacity usage.	40
6	Operational workload visuals showing transaction risk distribution and daily review load.	41
7	Prioritized review queue showing transactions routed to manual review with their risk level, amount, risk score, thresholds, and RL action.	41
8	Transaction action panel showing selected transaction details and the available analyst decisions.	42
9	Decision override form for updating a transaction decision and recording the reason in the audit trail.	42
10	High-risk transactions table showing transactions with high risk scores, thresholds, RL action, and final status.	43
11	Risk Insights visuals showing threshold behavior and card-group risk patterns.	43
12	Validation profit across training episodes for five independent seeds. The seeds converge to a tight band (spread of approximately \$22 on validation), confirming stable, reproducible training.	48
13	Offline Evaluation model performance view showing summary metrics and confusion matrix results.	50
14	Financial outcome comparison between no fraud management, the proposed Hybrid AE+CatBoost+RL system, and the oracle reference.	51
15	Financial Evaluation visuals showing caught fraud amount, missed fraud amount, and cost components.	51
16	Policy comparison between global thresholding, RL amount-conditioned policy, and deployed capacity-aware policy.	52
17	Policy threshold comparison showing the lower and upper decision thresholds for the evaluated policies.	52

List of Tables

1	Dashboard views and their purpose.	35
2	Dashboard module structure.	39
3	CNN and ANN test-set performance.	46
4	Summary of model results on the held-out test set.	47

List of Abbreviations

AE	Autoencoder
AUC	Area Under the Curve
CNN	Convolutional Neural Network
DQN	Deep Q-Network
FN	False Negative
FP	False Positive
FPR	False Positive Rate
IEEE-CIS	IEEE Computational Intelligence Society
KDE	Kernel Density Estimation
LOF	Local Outlier Factor
LTV	Customer Lifetime Value
MDP	Markov Decision Process
ML	Machine Learning
PG	Profit Gain
PPO	Proximal Policy Optimization
PR-AUC	Precision-Recall Area Under the Curve
ReLU	Rectified Linear Unit
RF	Random Forest
RL	Reinforcement Learning
ROC	Receiver Operating Characteristic
TN	True Negative
TP	True Positive

1 Chapter 1: Introduction

1.1 Background and Motivation

Financial Fraud, and especially credit card fraud has become critical challenge in modern digital economies. With the rapid expansion of online transactions, e-commerce platforms, and contactless payments, the volume of financial data processed daily has increased exponentially. This led to fraudsters using ever-evolving method that traditional rule based systems will not be able to detect which may come at the cost of rejecting legitimate transfers, or allowing fraudulent ones. Most ML-based systems also lack the dynamic nature of fraud techniques, which is why we got motivated to improve ML-based systems by also including an RL agent on top of it to maximise fraud detection. This approach aims to minimize financial loss while dynamically balancing fraud detection accuracy and operational costs.

1.2 Problem Statement

Our paper aims to address Credit Card Fraud for financial institutions through a machine learning and reinforcement learning model that is integrated into a dashboard that provides financial experts with the required statistics.

1.3 Project Objectives

The main goal of this paper is to minimize cost spent for financial institutions while maintaining very high fraud-detection accuracy. In addition, our goal is to integrate our model into an easy to use dashboard that helps financial experts deal with transactions.

1.4 Scope and Limitations

Our study is only limited to credit card fraud detection, therefore, other types of bank transactions, loan approvals, and any other type of financial fraud is not covered in our study.

One of the limitations of the IEEE- CIS Dataset is that a lot of the feature names were hidden which affected our ability to preprocess effectively. Also, Our approach assumes that manual financial Reviews are always done correctly.

1.5 Significance of Study

Financial fraud in credit card systems represents a significant and escalating global economic challenge. According to Statista, global losses from payment card fraud (including credit cards) have grown steadily over the past decade, reaching tens of billions of U.S. dollars annually, with forecasts indicating continued growth through 2028 [1]. This persistent rise highlights the inability of traditional fraud detection systems to adequately adapt to evolving fraud strategies. Also, Global payment infrastructure now supports billions of cards in circulation worldwide, reflecting the massive user base exposed to potential fraud risks [2]. Furthermore, Credit card payments constitute a substantial portion of global digital transactions, with transaction values reaching into the trillions of U.S. dollars annually and continuing to grow as e-commerce and fintech adoption accelerate worldwide [3].

This highlights the significant gap that our thesis tries to address. The beneficiaries of this research include banks, payment processors, fintech companies, and consumers, all of whom are directly affected by fraud-related losses and transaction disruptions. From an academic perspective, this study contributes to the advancement of intelligent fraud detection by demonstrating how reinforcement learning can be integrated with supervised learning models in a dynamic, high-risk financial environment.

Ultimately, this study addresses a large-scale, real-world problem with measurable economic consequences and proposes a scalable, adaptive solution aligned with the future trajectory of global digital payments.

1.6 Thesis Organization

The literature Review will be discussed first in chapter 2, then our methodology for the machine learning and RL cost function will be discussed in chapter 3, then our implementation will be discussed in chapter 4, brief discussion of Results in chapter 5 and possible future considerations in chapter 6.

2 Chapter 2: Literature Review

2.1 Introduction to Literature Review

Credit card fraud detection represents a complex intersection of highly imbalanced data classification and constrained operational decision-making. Developing an effective detection system requires not only identifying anoma-

lous patterns within massive transaction volumes but also doing so within the strict capacity limits of human review teams.

This literature review explores the foundational themes and recent advancements necessary to construct a robust, automated decision intelligence framework. The review is organized around two primary challenges:

Handling Extreme Class Imbalance and Noise: Exploring methodologies that move beyond traditional classification, specifically focusing on how deep learning architectures—such as autoencoders—can extract robust features from imbalanced datasets. **Operational Reality and Decision Intelligence:** Examining the gap between academic performance metrics and real-world constraints, including verification latency, concept drift, and sample selection bias driven by the alert-feedback loop. By analyzing these areas, this review establishes the necessity for a hybrid approach. It highlights why combining advanced anomaly detection networks with cost-sensitive dynamic decision strategies is essential for modern financial security systems.

2.2 Previous Research and Studies

Recent research in credit card fraud detection has increasingly focused on moving away from static, traditional classifiers toward more dynamic and structurally complex models. A significant portion of this research attempts to resolve the inherent imbalance of financial datasets. Jiang, Zhang, and Zou (2019) [7] addressed this specific challenge by proposing a hybrid deep learning pipeline to detect fraudulent transactions in highly skewed environments. The authors noted that traditional models often achieve artificially high accuracy by defaulting to the majority "normal" class, thereby failing at their primary objective: achieving a high recall rate for the minority fraud class. To counter this, they developed a three-stage framework utilizing a Denoising Autoencoder (DAE). First, the Synthetic Minority Oversampling Technique (SMOTE) was applied to balance the dataset. Because oversampling inherently introduces statistical noise, the data was subsequently processed through a 7-layer DAE. The DAE was explicitly trained on corrupted data to learn robust, underlying feature representations by reconstructing the original input. When the denoised features were passed to a deep neural network classifier, the proposed model significantly outperformed baseline methods, demonstrating that autoencoders are highly effective at mitigating the noise introduced by synthetic balancing techniques and dramatically improving minority-class detection rates.

2.2.1 Random Forest for Fraud Detection

Jonnalagadda et al. (2019) framed fraud detection as a supervised binary classification problem, where transactions are labeled as either fraudulent or legitimate and a Random Forest model learns to distinguish between the two from historical data. Their core argument is that Random Forest outperforms a single decision tree by training multiple trees on random subsets of the data and combining their outputs, which corrects the tendency to overfit and produces a more generalizable model.

The paper highlights several practical strengths of the algorithm. Because each tree is trained independently on a bootstrapped sample and a random subset of features, the model is both stable and scalable — adding a new data point affects only one tree rather than the entire model. It also handles mixed data types and missing values without requiring heavy preprocessing, which is a practical advantage when dealing with real- world transaction data.

The authors do note that prediction speed decreases as the number of trees grows, and that performance still depends on the quality of preprocessing and feature selection. They also acknowledge that accuracy alone is insufficient in fraud detection because of class imbalance, since a model that predicts everything as legitimate can still appear highly accurate.

These points directly shaped our implementation. We trained a Random Forest on the labeled IEEE- CIS dataset and handled class imbalance through balanced class weighting rather than relying on accuracy as the primary metric. Evaluation was based on precision, recall, F1- score, and AUC-ROC, which give a more meaningful picture of how well the model detects the minority fraud class.

2.2.2 Local Outlier Factor for Fraud Detection

Local Outlier Factor (LOF), introduced by Breunig et al. [9], is a density-based anomaly detection algorithm that identifies outliers by comparing local density of a point to its k - nearest neighbors. Unlike global methods, LOF can detect local anomalies — points that are anomalous relative to their immediate neighborhood even if they appear normal globally.

LOF algorithm:

1. Find the K nearest neighbors of every datapoint.
2. For any given point A calculate the Reachability- distance of that point with every other point in its neighborhood.
3. Using the calculated Reachability- distances for each point calculate the point's Local Reachability-Density.
4. The Local Outlier Factor of a point is calculated using the Local Reachability Density

of the point and its K nearest neighbors.

1. K-Nearest Neighbors of point A The set of the k nearest neighbors of point A , where $d_k(A)$ is the distance from A to its k -th nearest neighbor:

$$N_k(A) = \{B \mid d(A, B) \leq d_k(A)\}$$

2. Reachability Distance of A with respect to B The reachability distance of point A from point B , where $d_k(B)$ is the k -distance of B :

$$reach_dist_k(A, B) = \max\{d_k(B), d(A, B)\}$$

3. Local Reachability Density of point A The inverse of the average reachability distance of A from its k nearest neighbors:

$$rd_k(A) = \left(\frac{\sum_{B \in N_k(A)} reach_dist_k(A, B)}{|N_k(A)|} \right)^{-1}$$

4. Local Outlier Factor of point A The average ratio of the Local Reachability Densities of A 's k nearest neighbors to that of A itself:

$$LOF_k(A) = \frac{1}{|N_k(A)|} \sum_{B \in N_k(A)} \frac{rd_k(B)}{rd_k(A)}$$

Papers on LOF usage for fraud detection:

Maniraj et al. [10] applied LOF to credit card fraud detection, demonstrating its effectiveness as an unsupervised baseline. The algorithm requires no labeled fraud data during training, learning only the distribution of legitimate transactions. Fraudulent transactions are identified as points lying in sparse regions where local density is significantly lower than neighboring points.

Sugidamayatno and Lelono [11] compared LOF against two other outlier detection algorithms — INFLO (Influenced Outlierness) and AFV (Average Fuzzy Value) — on credit card transaction data. Their experiments showed that LOF achieved the highest accuracy (96%), outperforming INFLO (84%) and AFV (77%). The study also found that LOF was more efficient in detecting local outliers, which are characteristic of fraudulent transaction patterns, while INFLO and AFV struggled with locally dense fraud clusters.

However, LOF has well-known limitations: it struggles with high-dimensional data (the curse of dimensionality), is sensitive to the choice of the

hyperparameter k , and its computational complexity ($O(n^2)$) limits scalability. These limitations have motivated research into approximate methods and feature reduction techniques.

2.2.3 Kernel Density Estimation for Fraud Detection

Ram and Gray [8], in their work presented at the KDD Workshop on Anomaly Detection in Finance, framed fraud detection as a density- based anomaly detection problem. Their main idea is that fraudulent transactions tend to occur in low- density regions of the feature space, since they are rare events compared to the bulk of normal activity. Based on this, they positioned Kernel Density Estimation (KDE) as a natural baseline for fraud detection: any transaction that falls in a region of low estimated density is treated as a likely fraud.

While the framing is intuitive, the paper also points out several limitations that come with applying KDE directly. First, fitting a single density over the whole dataset throws away the label information, so the model loses the ability to separate the two classes cleanly. Second, the choice of bandwidth has a huge effect on performance: too small and the density becomes spiky and overfits to individual training points, too large and it gets washed- out and the fraud and legit regions blur into each other. Third, KDE struggles in high- dimensional settings because the density estimates become unreliable as the number of features grows.

These gaps directly shaped how we used KDE in our work. Instead of fitting one density on all transactions, we fit two class- conditional KDEs, one on the legitimate class and one on the fraud class, and score each transaction using the log- posterior so that the label information is actually used. To deal with the bandwidth issue, we ran a per- class grid search on the validation set rather than picking a single global value. And to limit the curse of dimensionality, we applied feature selection during preprocessing so that the KDE only sees a reduced set of informative features. This way the KDE setup keeps the simplicity and interpretability that Ram and Gray highlight, while addressing the main weaknesses they identified.

2.2.4 Convolutional Neural Network (CNN)

An important study in the advancement of deep learning for financial security is the research conducted by Alarfaj et al. (2022), titled "Credit Card Fraud Detection Using State- of- the- Art Machine Learning and Deep Learning Algorithms." This work focuses on the limitations of traditional machine learning in handling the extreme class imbalance and the complexity of fraud

transactions.

1D CNN Algorithmic Foundation The study shifts from standard image-centric 2D CNNs, proposing a 1D Convolutional Neural Network (1DCNN) architecture specifically made for temporal or sequential transaction data.

In this framework, the kernel filters (sizes 32×2 and 64×2) scan feature columns top- to- bottom through a sequence of transactions. This mechanism allows the model to perform feature reduction and identify fraud patterns without the need for the heavy manual pre- processing required by algorithms like SVM or Logistic Regression.

Evaluation of Layer Depth (14, 17, and 20 Layers) A core component of this research is the evaluation of network depth. The authors evaluated three architectures to determine the optimal configuration for fraud detection:

14-Layer CNN: Composed of two convolutional stages, batch normalization, and three hidden dense layers (units of 100, 50, and 25).

17-Layer CNN: Expanded the model with a third convolutional stage (64 kernels) and additional normalization/dropout layers.

20-Layer CNN: The most complex architecture, using four convolutional layers with increasing complexity to extract deeper feature representations.

Initial testing revealed a surprising efficiency in the 14- layer standalone model, which achieved 96.34% accuracy, slightly outperforming the 17 and 20- layer standalone units in speed and accuracy at 100 epochs.

Addressing Class Imbalance and the "Unseen Data" Challenge The researchers utilized the European credit card benchmark dataset, where fraud represents only 0.172% of 284,807 transactions.

To overcome this skewness, they performed experiments by balancing the data (removing non- fraudulent instances) during the training phase to prioritize minimizing the false negative rate, the highest priority for banks to minimize financial loss.

However, the study gives a critical insight into real- world application: while balancing improves training performance on known patterns, the accuracy significantly decreased on the unseen data.

The authors found that as the class imbalance increases, the ability to catch fraud on unseen data improves with deeper architectures. So, they concluded that the 20- layer CNN combined with a baseline model was the

”top method,” achieving a peak detection accuracy of 99.72% on balanced data.

Comparison with Traditional Benchmarks The Alarfaj et al. (2022) paper established a baseline by testing standard algorithms on the raw imbalanced data. Although models like Random Forest and Decision Trees reported high accuracies ($\sim 99.94\%$), the authors argue these figures are misleading because the models often become biased toward the majority class.

In contrast, the 1DCNN baseline validation accuracy on imbalanced data was 95.8% demonstrating that while its score might appear lower than biased ML, its ability to learn and generalize complex fraud patterns provides a clear advantage for long- term system robustness.

2.2.5 RL Cost Function

One of the most notable contributions to financial fraud detection was from Alejandro Correa Bahnsen et al [4] where a Bayes minimum risk classifier was used to predict expected financial loss which was a major step from predicting classification error. Also, Hoppner et al [5] laid the foundation for an instance- dependent cost- sensitive framework which emphasizes that fraud costs are not uniform across transactions but vary depending on contextual features. This significantly improves realism in fraud modeling by incorporating transaction- level heterogeneity into the learning process.

More recently, [6] extended cost- sensitive frameworks to trinary decision systems, where possible actions are accepting, rejecting, or reviewing a given transaction. Their approach integrates business- driven parameters such as customer lifetime value and review costs, enabling a more comprehensive assessment for models in a business use context. However, their framework assumes deterministic outcomes for manual review and relies on supervised learning for optimization.

2.3 Current State of the Field

Financial fraud detection in production environments has largely converged on supervised machine learning, specifically gradient-boosted decision trees (XGBoost, LightGBM, CatBoost) and Random Forests, due to their strong performance on tabular transaction data, inherent handling of mixed feature types, and interpretability via feature importance [12, 13]. Major payment processors such as Stripe, Adyen, and PayPal publicly report using ensemble tree-based models trained on hundreds of engineered features, with typical

retraining cycles ranging from daily to weekly to partially mitigate concept drift [14]. However, these systems almost exclusively output a binary accept/reject decision based on a fixed probability threshold, often calibrated to meet a target false positive rate (e.g., 0.5%–2% of transactions flagged for review).

Despite the maturity of supervised methods, three critical limitations persist in current practice.

First, optimization for accuracy or even ROC-AUC does not align with business costs. A fraud detection model that achieves 0.95 ROC-AUC may still be suboptimal if it frequently blocks low-amount legitimate transactions (high false positive cost) or misses high-amount frauds (high false negative cost). Bahnsen et al. [4] demonstrated that cost-sensitive classifiers can reduce total fraud-related losses by 30–50% compared to accuracy-optimized models, yet cost-sensitive deployment remains rare outside academic benchmarks. Industry practitioners often resort to post-hoc threshold tuning on a cost curve rather than integrating cost directly into training or decision logic [15].

Second, concept drift is handled reactively, not proactively. Fraudsters continuously adapt their tactics — changing transaction amounts, merchant categories, device fingerprints, and temporal patterns. A model that performs well in January may see its recall drop by 10–15 percentage points by March if not retrained [16]. Current solutions rely on periodic retraining or online learning with sliding windows, but these approaches do not explicitly model the drift nor optimize retraining schedules. Reinforcement learning has been proposed for dynamic threshold adjustment under drift [17], but to our knowledge no production system uses RL for this purpose.

Third, the review (deferral) action is underutilized. Most systems treat fraud detection as a binary choice, forcing every transaction into accept or reject. In reality, financial institutions employ teams of human reviewers who can examine flagged transactions, contact cardholders, or request additional verification. Yildirim et al. [6] showed that a trinary decision space (accept, reject, review) with a small review cost can significantly reduce the total cost compared to any binary threshold policy. However, few deployed systems implement dynamic review routing based on uncertainty estimates or expected cost; manual reviews are typically triggered by a simple score threshold, leading to either overloaded review queues or missed fraud cases.

Reinforcement learning, particularly Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN), remains almost entirely absent from operational fraud detection pipelines. The barriers are well-documented: reward signals in fraud are delayed (chargebacks arrive days or weeks after a transaction), off-policy evaluation is difficult due to selection bias (only reviewed

transactions have ground truth), and the high cost of exploring incorrect actions (e.g., accepting a large fraud) makes online training risky [18]. Recent academic work has attempted to bridge this gap using historical data with importance sampling or by training RL agents in simulated environments calibrated to real transaction streams [19]. Our work follows this direction but differs in two key ways: we use a trained hybrid AE+CatBoost classifier as a fixed risk scorer and train the PPO agent entirely offline on already-labeled historical data, and we design a cost function that explicitly includes a dynamic review cost to prevent over-deferral. This makes our approach practical for near-term deployment rather than purely theoretical.

2.4 Theoretical Framework

The design of our hybrid fraud detection system rests on three complementary theoretical pillars: **cost-sensitive learning**, **anomaly detection theory**, and **reinforcement learning for sequential decision-making under uncertainty**. Each provides formal grounding for a specific component of our architecture.

2.4.1 Cost-Sensitive Learning

Traditional supervised learning minimizes the expected misclassification error, i.e., $\mathbb{E}[\mathbf{1}_{\hat{y}_i \neq y_i}]$, which assigns equal weight to false positives and false negatives. In fraud detection this is inappropriate because the cost of accepting a fraudulent transaction (false negative) is typically the full transaction amount plus chargeback fees, while the cost of rejecting a legitimate transaction (false positive) is the lost profit margin. Formally, let $C(y_i, \hat{y}_i)$ be the cost incurred when the true label is y_i ($0 = \text{legitimate}$, $1 = \text{fraud}$) and the predicted action is $\hat{y}_i \in \{0, 1\}$ (accept/reject). A Bayes minimum risk classifier selects the action that minimizes the expected cost [4]:

$$\hat{y}_i = \arg \min_{a \in \{0,1\}} \mathbb{E}[C(y_i, a) \mid x_i] = \arg \min_a [P(y_i = 0 \mid x_i) \cdot C(0, a) + P(y_i = 1 \mid x_i) \cdot C(1, a)]$$

When $C(1, 0)$ (false negative cost) and $C(0, 1)$ (false positive cost) are constants, this reduces to thresholding the posterior probability $P(y_i = 1 \mid x_i)$ at a value determined by the cost ratio. However, fraud costs are not constant across transactions: a fraudulent \$10 transaction incurs far less loss than a fraudulent \$10,000 transaction. Hoppner et al. [5] extended the framework to **instance-dependent costs**, where $C(y_i, \hat{y}_i, x_i)$ varies with transaction features such as amount, cardholder history, and merchant risk.

Our cost function (Section 3.5) adopts this instance-dependent view, with the false negative cost set to $flm \times A_i$ (where flm amplifies the raw amount by chargeback and overhead multipliers) and the false positive cost set to $pr \times A_i \times ltv$ (lost profit scaled by customer lifetime value). This ensures that the model’s decisions are economically rational at the transaction level.

2.4.2 Anomaly Detection Theory

Anomaly detection provides a complementary view: fraudulent transactions are outliers relative to the distribution of legitimate behavior. Formally, let $p_{\text{legit}}(x)$ be the probability density of legitimate transactions in feature space. A point x is anomalous if $p_{\text{legit}}(x) < \epsilon$ for some small ϵ . This density-based approach underpins both Kernel Density Estimation (KDE) and Local Outlier Factor (LOF). KDE estimates $p_{\text{legit}}(x)$ non-parametrically using a kernel function — typically Gaussian — and a bandwidth parameter that controls smoothness [8]. The fraud score can be expressed as a log-posterior ratio:

$$s(x) = \log p(x|\text{fraud}) + \log \pi_{\text{fraud}} - \log p(x|\text{legit}) - \log \pi_{\text{legit}}$$

where π are class priors. A positive score indicates fraud is more likely than legitimacy given the observed features. LOF, by contrast, does not estimate a global density but compares the local density of a point to the average density of its k nearest neighbors [9]. Points with LOF significantly greater than 1 lie in sparse neighborhoods and are flagged as outliers.

A different but equally powerful anomaly detection paradigm is **reconstruction-based**. An autoencoder, trained exclusively on legitimate transactions, learns a low-dimensional bottleneck representation that captures the intrinsic manifold of normal behavior. The reconstruction error $\|x - \hat{x}\|^2$ is small for legitimate transactions (they lie on the learned manifold) and large for frauds (they do not reconstruct well). Jiang et al. [7] showed that adding a denoising autoencoder to oversampled fraud data improves robustness to synthetic noise. Our hybrid model combines reconstruction error (unsupervised) with supervised CatBoost probabilities, benefiting from both sensitivity to novel fraud patterns (autoencoder) and discriminative power on known fraud types (CatBoost). Notably, the two components run in parallel rather than as a stacked pipeline — each produces an independent score and the two are blended — so the supervised model never depends on the autoencoder’s representation.

2.4.3 Reinforcement Learning for Cost Optimization

Reinforcement learning frames fraud detection as a sequential decision process with a well-defined reward signal. At each transaction i , the agent observes a state $s_i = (f_i, \mathbf{z}_i)$, where f_i is the risk score from the hybrid AE+CatBoost classifier and $\mathbf{z}_i$ includes contextual features such as transaction amount, time since last transaction, and merchant category. The agent selects an action $a_i \in \{\text{accept, reject, review}\}$. After the action, the environment reveals the true label y_i (for reviewed transactions) or the eventual chargeback outcome (for accepted transactions), and the agent receives a reward $r_i = -C(a_i, y_i, x_i)$ where C is the instance-dependent cost function from Section 3.5.

We use **Proximal Policy Optimization (PPO)** [20] as the RL algorithm. PPO is a policy gradient method that optimizes a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta)$ is the probability ratio between the new and old policies, $\hat{A}_t$ is the advantage estimate, and ϵ is a hyperparameter (typically 0.2) that prevents destructively large policy updates. PPO is preferred over DQN in our setting because the action space is small but the state space is continuous, and PPO handles stochastic policies naturally — important when the optimal decision near the boundary between accept and review is probabilistic. The thresholds T_L and T_H (accept if $f_i < T_L$, reject if $f_i > T_H$, review otherwise) can be learned either explicitly as policy parameters or implicitly via a neural network policy that maps states to action probabilities. We choose the latter to allow for non-threshold-based policies (e.g., reviewing high-amount transactions even when the risk score is moderate), though our initial implementation parameterizes the policy as a soft threshold function for interpretability.

The theoretical guarantee of this framework is that, given sufficient exploration and a well-specified cost function, the PPO agent converges to a policy that minimizes the expected cumulative financial loss over the transaction stream, subject to the constraint that manual review capacity is respected through the dynamic review cost C_{Review} . This transforms fraud detection from a classification problem into a business-optimization problem — a shift that we argue is both practically necessary and theoretically well-grounded.

2.5 Research Gap

To the best of our Knowledge, no paper has explored using RL as an optimizer on top of Machine Learning, and papers usually assess the performance of the model while neglecting its outcome on the business side, and vice versa. Our approach aims to bridge that Gap and provide a hybrid architecture that is seamlessly integrated, with comprehensive model assessment.

3 Chapter 3: Methodology

3.1 System Overview

3.1.1 Initial System Design

Our Entire system architecture is shown here:

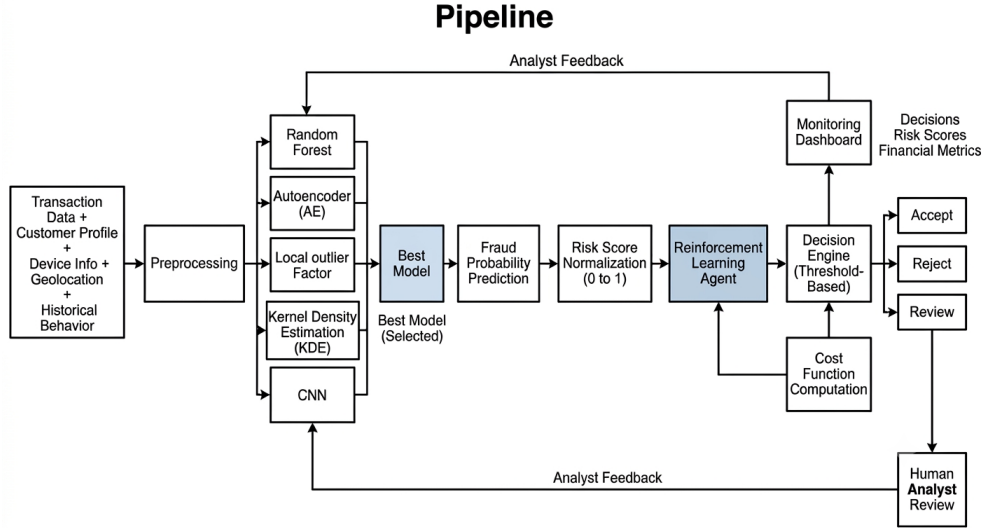


Figure 1: Hybrid ML-RL Fraud Detection System Architecture

We will first start with our dataset, and after applying the preprocessing discussed in chapter 2, we will test out the ML models and pick the best performing one to be then passed on to the RL after normalizing its output to be a risk score from 0 to 1, then the RL agent will optimize these thresholds after computing the cost function and we will then use these thresholds for our trinity decisions (accept, Review, Reject), and we will leave the transactions pending review for the financial expert. While this design established the core pipeline, it exhibited several limitations that motivated a substantially revised architecture. The classifier selection was based on statistical

performance metrics (AUC) rather than financial objectives, the cost function did not account for transaction amount heterogeneity, and the RL agent operated on a single pair of fixed thresholds that could not adapt to the varying risk profile of transactions at different value levels. These limitations motivated the final system design described below.

3.1.2 Final System Design

The final system architecture — illustrated in Figure 2 — addresses the limitations of the initial design through three principal changes: a hybrid classifier that directly optimises for financial performance, an extended cost function that models all eight decision-outcome combinations, and an amount-aware RL policy that adapts decision thresholds continuously with transaction value.

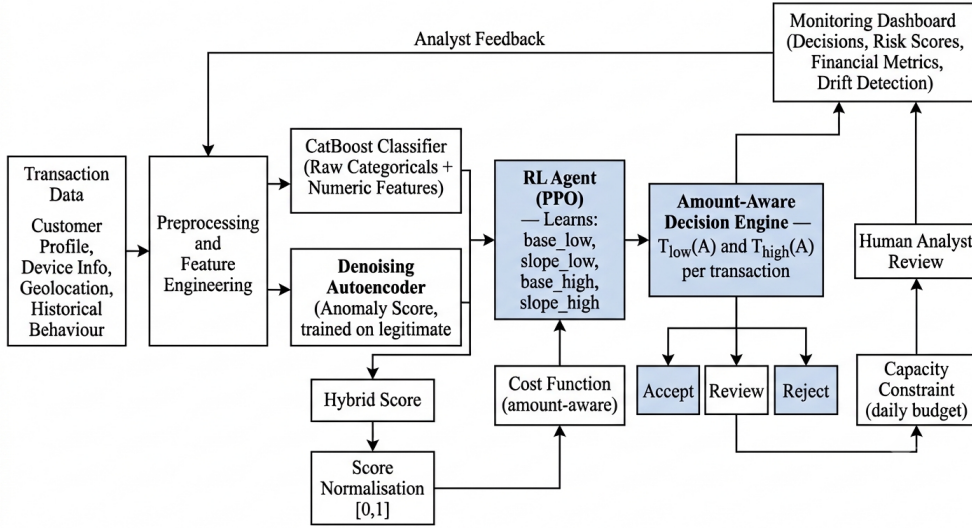


Figure 2: Final Amount-Aware Hybrid ML-RL Fraud Detection System Architecture

The pipeline proceeds as follows. After preprocessing, two models are trained in parallel: a CatBoost classifier operating natively on raw categorical features combined with engineered numeric features, and a shared Autoencoder trained exclusively on legitimate transactions to produce an anomaly percentile score. The outputs of these two components are blended into a single hybrid fraud score using a weighting parameter α selected to maximise validation-set profit rather than AUC:

$$\text{score} = \alpha \cdot \text{CatBoost}_{\text{mm}} + (1 - \alpha) \cdot \text{AE}_{\text{pct}}$$

This hybrid score is normalised to $[0, 1]$ and passed to the RL agent. Rather than learning two fixed scalar thresholds, the agent learns four parameters $\{base_{low}, slope_{low}, base_{high}, slope_{high}\}$ that define per-transaction thresholds varying continuously with normalised log-amount:

$$T_{low}(A_i) = base_{low} + slope_{low} \times z(A_i), \quad T_{high}(A_i) = base_{high} + slope_{high} \times z(A_i)$$

where $z(A_i)$ is the standardised log-amount computed using training-set statistics. The Decision Engine applies these thresholds to each transaction individually, routing it to Accept, Review, or Reject. A capacity constraint on the Review channel enforces a maximum daily review volume consistent with the available analyst headcount. Transactions that exceed this budget are resolved via a profit-optimal binary fallback rule. The Monitoring Dashboard aggregates decisions, risk scores, and financial metrics in real time, and analyst feedback on reviewed transactions is used to monitor for distributional drift over time.

3.2 Data Collection and Preprocessing

The system was developed and evaluated using the IEEE Fraud Detection Dataset, which contains a massive volume of real-world e-commerce transactions. The raw data consists of a primary transaction dataset (590,540 records, 394 features) and an associated identity dataset (144,233 records, 41 features), which were merged via a unique Transaction ID to create a unified dataset of 434 initial features. Exploratory Data Analysis (EDA) confirmed an extreme class imbalance, with a baseline fraud rate of only 3.50%. To ensure realistic evaluation and prevent data leakage, the dataset was strictly split chronologically into a 70% training set, 15% validation set, and 15% testing set. The preprocessing pipeline included several vital steps:

Missing Value Imputation: Features exhibiting more than 50% missing values were analyzed for fraud signals, and numeric features were filled using training-set medians. **Feature Engineering:** Time-aware, past-only aggregate features (such as past transaction counts and mean amounts per card) and relative deviation features were engineered to capture behavioral shifts without introducing future leakage. **Categorical Encoding:** Low-cardinality categorical variables were transformed using a combination of frequency encoding and top-K one-hot encoding. **Feature Selection & Scaling:** Highly correlated and redundant features were pruned using a maximum correlation threshold of 0.92, yielding 320 variables. A subsequent adversarial-validation drift audit found that four card-history aggregate features—the past transaction counts and past mean amounts keyed on `card1` and `card2`—carried

strong temporal drift between the validation and test periods (adversarial AUC of approximately 0.97 with these features present versus 0.71 without). Because such drift carrying features inflate apparent performance while degrading generalisation to the later test period, all four were dropped from the entire pipeline, leaving a final feature set of 316 variables. Continuous variables were then scaled with a StandardScaler fitted on the training split, and the denoising autoencoder was trained on the legitimate-only subset of that scaled training data.

3.3 System Design and Constraints

The system architecture was developed iteratively, beginning with a purely unsupervised baseline and evolving into a complex hybrid framework to maximize detection robustness under strict operational constraints.

3.3.1 Autoencoder Approach

Phase 1: The Autoencoder Ensemble Baseline Initially, the system relied solely on an unsupervised Autoencoder Ensemble to detect anomalies. Because relying on a single network can make a system vulnerable to specific statistical noise, an ensemble of three diverse denoising autoencoders (AE_01, AE_02, AE_03) was implemented.

Network Topologies: The members featured varying hidden layer structures (e.g., from a 256-128 structure to a deeper 384-192-96 structure) and differing bottleneck constraints to capture multiple levels of latent abstraction. **Scoring:** Each model was trained exclusively on normal transactions and generated independent reconstruction errors for incoming data. These scores were blended using weights derived from each member’s PR- AUC performance on the validation set. **Limitations:** While the ensemble successfully identified anomalies without relying on labels, its purely unsupervised nature limited its precision in distinguishing between highly irregular legitimate spending and actual fraud.

Phase 2: The Hybrid Autoencoder + CatBoost (Final Classifier)

To bridge the gap between anomaly detection and high-precision classification, the system was upgraded to a hybrid architecture that runs two complementary detectors **in parallel** and blends their outputs. Unlike a stacked pipeline, neither model consumes the other’s representation.

Unsupervised path. A single denoising autoencoder (with a 256–128 hidden structure and a 48-dimensional bottleneck constrained by L_1 activity regularization) was trained exclusively on legitimate transactions. For each

transaction, a reconstruction-error anomaly score was computed as a weighted combination of the mean, top- k , and maximum squared reconstruction error, then converted to a percentile against the legitimate-transaction reference distribution.

Supervised path. Independently, a CatBoost gradient-boosting classifier (800 iterations, depth 6, learning rate 0.03, balanced class weights) was trained on the raw engineered feature set—native categorical features together with the numeric features. CatBoost was deliberately left uncalibrated to preserve the natural spread of its probability outputs, on which the downstream RL agent relies. Its probabilities were passed through a square-root transform followed by min-max scaling to spread the score distribution away from a degenerate near-zero cluster.

Decision blending. The final risk score was formed as a convex combination of the two scores,

$$s_{\text{hybrid}} = \alpha \cdot s_{\text{CatBoost}} + (1 - \alpha) \cdot s_{\text{AE}},$$

with the blend weight $\alpha = 0.95$ selected not by AUC but by **validation profit**, directly optimizing the business objective the system targets.

Design constraints. A primary operational constraint of the system is the limited daily review capacity of human fraud investigators. This constraint is not enforced by calibrating a fixed alerting threshold on the classifier itself; instead, the classifier is left uncalibrated and the routing of transactions into the accept, review, and reject zones is deferred entirely to the reinforcement-learning layer (Chapter 3.5) and its downstream capacity mechanism. This separation—an uncalibrated risk scorer feeding a cost-aware decision policy—is what allows the review budget to be treated as an adjustable operational parameter rather than a property baked into the classifier, and it is developed in full in the cost-function and policy sections that follow.

3.3.2 Random Forest Approach

The Random Forest model was implemented as a supervised binary classifier trained on the preprocessed IEEE- CIS dataset. The data was processed through the shared preprocessing pipeline, then split into training and test sets with stratification to maintain the fraud- to- legitimate ratio in both partitions. Each tree was trained on a bootstrapped sample of the training data and a random subset of features, and final predictions were made by aggregating probability estimates across all trees.

Random Forest was selected because it requires no feature scaling, handles high- dimensional mixed- type data effectively, and is resistant to overfitting by design. The number of estimators and maximum tree depth were tuned

to balance performance against computational cost. Class imbalance was addressed using balanced class weighting, which increases the penalty for missing a fraud case during training and prevents the model from defaulting to the majority class.

3.3.3 LOF Methodology

Novelty detection mode was used to enable inference on new transactions. Manhattan distance was selected after comparison with Euclidean. Due to computational constraints, 50,000 normal transactions were sampled for training. The value $k = 20$ was selected through grid search on the validation set, balancing recall and precision. The contamination parameter was set to the validation fraud rate (0.0343).

Implementation Details:

- Training samples: 50,000 normal transactions
- Features: 320
- k (neighbors): 20
- Distance metric: Manhattan
- Contamination: 0.0343 (validation fraud rate)
- Training time: ~ 164 seconds

Threshold Selection:

Validation LOF scores were used to select the optimal threshold that maximizes the F1 score. Transactions with scores below this threshold are flagged as fraudulent.

3.3.4 KDE Approach

The KDE model was implemented as a supervised, class- conditional density estimator rather than the standard one- class baseline. Two Gaussian KDEs were fitted separately, one on the legitimate transactions and one on the fraud transactions, both using the same preprocessed and standard- scaled features produced by the shared feature- extraction pipeline. For each transaction, a fraud score was computed as the log- posterior:

$$\text{score}(x) = \log p(x \mid \text{fraud}) + \log \pi_{\text{fraud}} - \log p(x \mid \text{legit}) - \log \pi_{\text{legit}}$$

where π_{fraud} and π_{legit} are the class priors estimated from the training set. A transaction is then flagged as fraud whenever this score is greater than or equal to a threshold tuned on the validation split.

KDE is non-parametric and does not assume any specific shape for the fraud or legit distributions, which is a good fit for the IEEE-CIS data where the underlying structure is not well known. The most important hyperparameter is the bandwidth, since it directly controls how smooth the density is: a small bandwidth gives a spiky density that overfits the training points, while a large bandwidth blurs the separation between classes. Because of this, the bandwidth was tuned per class instead of using a single global value. A grid search over $\{0.2, 0.4, 0.6\}$ was performed for each class, evaluated on the validation set using the precision-recall curve. The final decision threshold was also picked on validation.

3.3.5 CNN Methodology

The CNN model was implemented as a supervised deep-learning baseline for fraud classification on the preprocessed IEEE-CIS transaction features. Since the transaction records are tabular rather than image-based, the feature vector was reshaped into a one-dimensional sequence so that Conv1D layers could scan local groups of neighboring features. This allowed the network to learn short-range feature interactions before passing the extracted representation to fully connected layers.

In this implementation, each transaction was represented using 209 preprocessed input features. For CNN compatibility, the input was reshaped from a flat feature vector into a tensor of shape $(209, 1)$. The CNN architecture consisted of two convolutional blocks followed by dense classification layers. The first convolutional block used 32 filters with kernel size 2, batch normalization, and dropout of 0.2. The second convolutional block used 64 filters with kernel size 2, batch normalization, and dropout of 0.5. The output was then flattened and passed through a dense layer of 64 neurons with dropout, followed by dense layers of 100, 50, and 25 neurons before the final sigmoid output neuron.

The CNN output is interpreted as the probability of the transaction being fraudulent. Since the task is binary classification, the model was trained using binary cross-entropy loss and the Adam optimizer. Class imbalance was handled using class weights so that fraud samples received a higher penalty during training. Early stopping was applied using validation AUC to reduce overfitting and restore the best-performing weights.

3.3.6 ANN Methodology

An Artificial Neural Network (ANN) was also implemented as a dense deep-learning baseline. Unlike the CNN, the ANN does not apply convolutional filters or reshape the feature vector into a sequence. Instead, it receives the 209 preprocessed features directly and learns nonlinear feature interactions through fully connected layers.

The ANN architecture consisted of three hidden dense layers. The first layer used 256 neurons with ReLU activation, batch normalization, and dropout of 0.3. The second layer used 128 neurons with ReLU activation, batch normalization, and dropout of 0.3. The third layer used 64 neurons with ReLU activation and dropout of 0.3. A final sigmoid neuron was used to produce the fraud probability.

The ANN used the same binary classification setup as the CNN. It was trained using binary cross-entropy loss, the Adam optimizer, class weighting for imbalance handling, and early stopping based on validation AUC. This made the ANN a useful comparison point for evaluating whether convolutional feature extraction improved performance over a simpler dense neural network.

3.3.7 Baseline Model Evaluation and Selection

The models previously discussed did not achieve a satisfactory result (refer to chapter 4), therefore Catboost was used as the supervised learning layer for this project working in parallel with a denoising autoencoder.

3.4 Evaluation Metrics

Because the dataset exhibits a severe class imbalance (96.5% normal vs. 3.5% fraud), standard accuracy is a misleading metric. A model could achieve 96.5% accuracy simply by classifying every transaction as normal. Therefore, the system performance was assessed using the following metrics:

- Precision- Recall AUC (PR- AUC): The primary metric used for evaluating and weighting the ensemble models, as it focuses specifically on the model's ability to identify the minority fraud class without being skewed by the massive number of true negatives.
- Recall (Sensitivity): Crucial for measuring the percentage of actual fraudulent transactions successfully caught by the system.
- False Positive Rate (FPR): Used to calibrate the decision threshold, ensuring the system does not overwhelm investigators with false alarms.

- Receiver Operating Characteristic AUC (ROC- AUC): Used as a secondary, general measure of the models' ability to rank fraudulent transactions higher than normal ones.

3.5 Cost Function Design

3.5.1 Initial Design

The foundation of our cost function is rooted in existing cost-sensitive fraud detection literature. Let $y_i \in \{0, 1\}$ denote the ground truth label, where 0 represents a legitimate transaction and 1 represents a fraudulent one. Let $\hat{y}_i \in \{0, 1\}$ denote the predicted label.

Confusion Matrix The interaction between ground truth and predicted label yields the standard confusion matrix:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} = \begin{bmatrix} y_i = 0, \hat{y}_i = 0 & y_i = 0, \hat{y}_i = 1 \\ y_i = 1, \hat{y}_i = 0 & y_i = 1, \hat{y}_i = 1 \end{bmatrix}$$

Building on the formulation of Höppner et al. and Bahnsen et al., which establish that fraud cost is transaction-dependent while operational cost is fixed, and the three-action extension of Yildirim et al., our initial cost function is defined as:

$$\text{TotalCost} = \sum_{i \in FN} (flm \times A_i) + \sum_{i \in FP} (pr \times A_i \times ltv) + \sum_{i \in \text{Review}} C_{\text{Review}}$$

where the decision rule governing which sum each transaction falls into is:

$$\text{Action} = \begin{cases} \text{Accept} & f_i < T_L \\ \text{Review} & T_L \leq f_i \leq T_H \\ \text{Reject} & f_i > T_H \end{cases}$$

Here $f_i = P(y = 1 \mid x_i)$ is the model's fraud probability score for transaction i , and T_L, T_H are the lower and upper decision thresholds learned by the RL agent.

The parameters are defined as follows:

- A_i : the transaction amount in dollars.

- *flm*: the fraud loss multiplier. It amplifies the financial damage of a missed fraudulent transaction beyond the raw amount A_i , capturing chargeback fees, dispute handling, legal costs, and processor penalties. The total loss when a fraud is accepted is therefore $flm \times A_i$.
- *pr*: the profit rate (e.g. 0.02 for a 2% margin). When a legitimate transaction is accepted, the merchant realises $pr \times A_i$ in gross profit. When a legitimate transaction is wrongly rejected, that same $pr \times A_i$ is forfeited.
- *ltv*: the customer lifetime value multiplier. It scales the cost of a false positive beyond the immediate transaction, reflecting the expected future revenue lost when a legitimate customer is incorrectly blocked.
- C_{Review} : the dynamic cost per manual review, defined as:

$$C_{\text{Review}} = rc \times \lambda, \quad \lambda = ao + rp$$

where rc is the fixed per-review cost, ao is a dynamic administrative overhead that increases with the volume of transactions pending review, reflecting queue pressure and staffing constraints, and rp is a fixed penalty term that discourages the RL agent from over-relying on the review channel.

The RL agent’s reward is the negative of the total cost:

$$R = -\text{TotalCost}$$

Observed Limitations Despite its theoretical grounding, this initial formulation exhibited two critical failure modes during training:

- **Amount-blindness.** The cost function treats a \$5 transaction and a \$5,000 transaction identically in terms of routing logic. Since the flat review cost rc is fixed while the fraud loss $flm \times A_i$ scales with amount, the financially optimal review threshold differs substantially across transaction sizes. Without this awareness, the agent converged to degenerate policies that placed all transactions into review or rejected them wholesale, as both extremes avoided the asymmetric penalties without requiring genuine discrimination.
- **No reward for accepting legitimate transactions.** The formulation only penalises the agent for bad decisions — missed fraud and

wrong rejections. It provides no positive signal for correctly accepting legitimate transactions, which constitute the vast majority of the dataset. As a result, the agent had no incentive to maintain a functioning accept zone, reinforcing collapse toward excessive review or rejection.

These limitations required a fundamental redesign of both the cost structure and the policy parameterisation.

3.5.2 Extended Cost Function

Before introducing the amount-aware policy, the cost function itself was extended to address the second limitation identified above. The full eight-outcome formulation models every combination of decision and ground truth explicitly.

For the Accept and Reject decisions, the costs are:

$$C_{\text{Accept}} = \begin{cases} flm \times A_i & y_i = 1 \quad (\text{fraud missed}) \\ -pr \times A_i & y_i = 0 \quad (\text{profit realised}) \end{cases}$$

$$C_{\text{Reject}} = \begin{cases} 0 & y_i = 1 \quad (\text{fraud correctly blocked}) \\ pr \times A_i \times ltv & y_i = 0 \quad (\text{legitimate customer wrongly blocked}) \end{cases}$$

The Review cost is modelled as a probability-weighted mixture of the Accept and Reject outcomes, governed by $prev$ — the probability that a manual reviewer reaches the correct conclusion:

$$C_{\text{Review}} = \begin{cases} rc + (1 - prev) \times flm \times A_i & y_i = 1 \\ rc + pr \times A_i \times [(1 - prev) \times ltv - prev] & y_i = 0 \end{cases}$$

The legitimate-review cost can be decomposed as:

$$\underbrace{rc}_{\text{flat fee}} + \underbrace{(1 - prev) \times pr \times A_i \times ltv}_{\text{incorrect-review penalty}} - \underbrace{prev \times pr \times A_i}_{\text{correct-review profit credit}}$$

This decomposition reveals that the Review action is structurally a mixture of Accept and Reject: with probability $prev$ the reviewer reaches the correct outcome, earning the same $pr \times A_i$ profit as an accept; with probability $1 - prev$ the reviewer errs, incurring the same $pr \times A_i \times ltv$ loss as a

reject; with a flat fee rc charged regardless. This ensures the cost function rewards the agent for correctly clearing legitimate transactions, rather than only penalising it for errors.

When $prev = 1.0$ (a perfect reviewer), the review cost for a legitimate transaction reduces to $rc - pr \times A_i$, meaning review is profitable whenever $pr \times A_i > rc$, i.e. for sufficiently large transactions. In practice $prev$ should reflect realistic reviewer accuracy (e.g. 0.85–0.90) to prevent the review channel from trivially dominating accept and reject.

3.5.3 Amount-Aware Policy

The decision boundaries that separate accept, review, and reject are not known in advance. Unlike the classifier, which is trained against ground-truth fraud labels, there is no labelled “correct threshold” to supervise: the optimal cutoffs depend on the full cost structure and vary with transaction amount, and no dataset provides them. We therefore frame threshold selection as a reinforcement-learning problem, in which an agent proposes candidate thresholds, observes the realised financial outcome, and adjusts toward higher profit. This is formulated as a Markov Decision Process: at each step the agent observes a *state* summarising the distribution of risk scores across a batch of transactions; it takes an *action* consisting of the four parameters that define the amount-conditioned thresholds; and it receives a *reward* equal to the realised profit under the cost function of Section 3.5.2—not classification accuracy. The agent thus optimises directly for the business objective, learning the decision boundaries that no label set could provide.

To address the amount-blindness identified in Section 3.5.1, we replace the two fixed scalar thresholds T_L and T_H with per-transaction thresholds that vary continuously with transaction amount. Rather than learning separate thresholds for discrete amount bands, the RL agent learns four scalar parameters:

$$\theta = \{base_{low}, slope_{low}, base_{high}, slope_{high}\}$$

and applies the following rule for each transaction i :

$$T_{low}(A_i) = base_{low} + slope_{low} \times z(A_i) \tag{1}$$

$$T_{high}(A_i) = base_{high} + slope_{high} \times z(A_i) \tag{2}$$

where $z(A_i)$ is the normalised log-amount:

$$z(A_i) = \frac{\log(1 + A_i) - \mu_{\log A}}{\sigma_{\log A}}$$

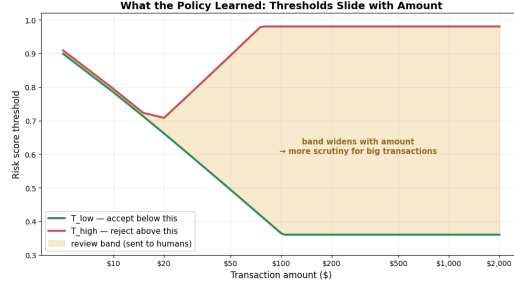


Figure 3: Learned decision thresholds as a function of transaction amount. The review band widens with amount, directing larger transactions toward human review.

and $\mu_{\log A}$, $\sigma_{\log A}$ are the mean and standard deviation of log-amounts computed on the training set and frozen thereafter, ensuring no test-time data leakage.

Interpretation of Parameters $base_{low}$ and $base_{high}$ are the accept/review and review/reject boundaries respectively, evaluated at the median transaction amount where $z = 0$. They play the same role as the fixed thresholds T_L and T_H in the initial design, but now serve as intercepts of a linear function rather than global constants.

$slope_{low}$ and $slope_{high}$ are the gradients of those boundaries with respect to normalised log-amount. They answer the question: *as transactions become larger, how should the decision boundaries shift?*

A negative $slope_{low}$ lowers the accept/review boundary for large transactions, meaning the agent becomes more willing to route expensive transactions to review even at modest suspicion scores. This is financially rational: for a \$500 transaction, the expected fraud loss at a 30% score is $0.30 \times flm \times 500 = \450 , which far exceeds the \$3.50 review cost. For a \$5 transaction at the same score, the expected fraud loss is \$4.50 — barely above the review cost — making review only marginally worthwhile.

A negative $slope_{high}$ lowers the review/reject boundary for large transactions, causing the agent to outright reject high-value suspicious transactions rather than sending them to review. This reflects the asymmetric cost structure: the cost of letting a large-amount fraudulent transaction through the review queue (if the reviewer errs) substantially exceeds the cost of wrongly rejecting a large legitimate transaction.

Connection to the Cost Function The amount-aware thresholds are a direct consequence of the cost function’s own amount-dependence. Even in

the initial formulation, there exists a transaction-specific lower bound below which review is never financially worthwhile:

$$T_{low}^{\min}(A_i) = \frac{rc}{flm \times A_i}$$

For very small A_i this bound approaches or exceeds 1.0, making review unprofitable at any suspicion level. The amount-continuous policy generalises this by allowing the agent to learn the full shape of the optimal boundary across the amount distribution, rather than relying on a single global threshold that cannot simultaneously be optimal for both a \$2 and a \$2,000 transaction.

The RL agent is trained to find the four parameters θ that maximise total profit across the dataset, with the per-transaction thresholds computed at inference time using the frozen statistics $\mu_{\log A}$ and $\sigma_{\log A}$. This produces a policy that is both more financially realistic and substantially more robust to the degenerate collapse observed in the initial design.

Algorithm and Environment The policy is optimised using Proximal Policy Optimization (PPO). PPO is chosen over value-based methods such as Deep Q-Networks because the action space here is continuous—the threshold parameters are real-valued—whereas DQN is designed for discrete actions. PPO’s clipped surrogate objective also provides stable updates without the sensitivity to step size that afflicts vanilla policy gradients, which matters given the noisy, profit-based reward signal. Two environment formulations were used. In the first, the agent sets a single set of parameters evaluated over the whole validation set at once; because the state is effectively constant, this reduces to a contextual-bandit problem rather than a full sequential decision process. In the deployed formulation, transactions are processed in chronological batches and the reward accrues sequentially, yielding a genuine sequential MDP. We report results from the batched sequential agent, which is the configuration used in deployment.

3.5.4 Review-Capacity Layer

The amount-aware policy of Section 3.5.3 assumes every transaction routed to review can actually be examined by a human. In practice, the number of manual reviews a fraud team can perform per day is strictly bounded. Left unconstrained, the learned policy routes roughly 25–27% of all transactions to review, far exceeding any realistic staffing level. We therefore introduce a capacity layer that operates on top of the frozen policy at evaluation time, translating a fixed daily review budget into the accept/review/reject stream.

Review Budget The review budget is expressed as a fraction of the transaction stream:

$$\text{budget_frac} = \frac{n_{\text{analysts}} \times r_{\text{day}}}{N_{\text{day}}},$$

where n_{analysts} is the number of human reviewers, r_{day} is the number of reviews each can perform per day, and N_{day} is the mean number of transactions per day. With approximately 2,878 transactions per day and 100 reviews per analyst, eight analysts correspond to a budget of roughly 27.8%.

Spending the Budget The capacity layer processes transactions in chronological order and spends the review budget through three stages:

1. **Grant in time order.** While budget remains, transactions the policy routes to the review band are granted a human review as they arrive.
2. **Triage under pressure.** As the remaining budget tightens, a pressure-gated rule diverts the cheapest borderline transactions away from the review queue so that scarce review slots are reserved for larger-amount transactions, where the expected financial stakes are highest. Concretely, the lower decision boundary is raised by a term $k \cdot \text{pressure} \cdot \text{relu}(-z(A_i))$, where the pressure factor grows as the fraction of budget consumed increases, and the coefficient k is selected on the validation set.
3. **Profit-optimal fallback.** Once the budget is exhausted, no further reviews are possible, and remaining transactions are decided by the policy that minimizes expected cost in the absence of review. Rejecting a transaction of fraud probability p is preferable to accepting it precisely when the expected fraud loss exceeds the expected lost profit, which yields the amount-independent threshold

$$p^* = \frac{pr(1 + ltv)}{flm + pr(1 + ltv)}.$$

With the parameters used in this work ($pr = 0.02$, $ltv = 5$, $flm = 3$), this evaluates to $p^* \approx 0.0385$: a budget-exhausted transaction is rejected if its fraud probability is at least 3.85% and accepted otherwise.

Separation of Training and Deployment A central design decision is that the capacity layer is **not** trained end-to-end with the policy. The PPO policy is trained once, unconstrained, and frozen; the capacity layer is then applied as a lightweight operational wrapper. When staffing changes, only

the static threshold offsets and the triage coefficient are re-derived for the new budget—a computation of seconds—while the underlying PPO policy is never retrained. This separation makes the review budget an adjustable operational dial rather than a property baked into the model. Attempts to instead train the policy directly under a fixed budget were found to generalize poorly, because the roughly two percentage-point distribution shift between the validation and test periods caused budget-coupled policies to violate the intended capacity on unseen data.

3.5.5 Dashboard and Human-in-the-Loop Decision Support

After applying the review-capacity layer, the final system output is presented through an interactive dashboard. The dashboard represents the human-in-the-loop layer of the proposed framework, where the outputs of the hybrid classifier, PPO policy, and capacity mechanism are converted into a practical workflow for financial experts. This stage is important because the project does not stop at producing a fraud score; it also shows how this score is used to support transaction-level decisions.

The dashboard follows the same decision structure used by the RL agent. Each transaction is assigned one of three actions: accept, review, or reject. Accepted transactions are treated as low-risk transactions, reviewed transactions are directed to the analyst queue, and rejected transactions are blocked by the system. This preserves the trinary decision logic of the RL model while making the output easier to inspect and act upon.

The dashboard is divided into two main views: **Bank Operations** and **Offline Evaluation**. The Bank Operations view shows how the system would be used by employees in practice. It focuses on the system risk scores, deployed decisions, review capacity, review queue, and analyst actions. The Offline Evaluation view is used to present the project results using the labeled IEEE-CIS dataset. Since the dataset contains fraud labels, this view can show evaluation results such as the confusion matrix, precision, recall, detected fraud, missed fraud, protected exposure, and financial comparisons.

Table 1: Dashboard views and their purpose.

Dashboard view	Purpose	Label usage
Bank Operations	Shows the employee workflow using the system risk scores, deployed decisions, review queue, capacity indicators, and analyst actions.	No fraud label used
Offline Evaluation	Presents project results using the labeled test data, including model performance, financial impact, and policy comparison.	Uses dataset labels

The Bank Operations view also includes a manual override mechanism. This is useful because analyst decisions may depend on information outside the transaction features, such as customer confirmation or internal investigation. Therefore, the analyst can approve, reject, or hold a transaction for investigation. Each action is recorded in an audit trail that stores the selected transaction, the original system decision, the final analyst decision, the reason for the action, and the timestamp.

This layer completes the system by connecting model output to analyst decision-making. The hybrid AE+CatBoost model produces the transaction risk score, the PPO policy assigns the cost-aware action, the capacity layer controls review workload, and the dashboard presents the final workflow to the financial expert.

4 Chapter 4: Implementation and Results

4.1 Implementation Details

4.1.1 Autoencoder Implementation

As described in section 3.3, the autoencoder ensemble and hybrid model were implemented with the specified topologies and training procedures. The ensemble used three denoising autoencoders with varying hidden structures (256-128, 384-192-96, etc.) and bottleneck constraints. The final hybrid model used a single autoencoder with a 256-128 hidden structure and encoding dimension 48, combined in parallel with a CatBoost classifier (800 iterations, depth 6) via profit-selected decision blending ($\alpha = 0.95$ on the CatBoost score).

4.1.2 Random Forest Implementation

The Random Forest model was implemented as a supervised binary classifier trained on the preprocessed IEEE- CIS dataset. The data was processed through the shared preprocessing pipeline, then split into training and test sets with stratification to maintain the fraud- to- legitimate ratio in both partitions. Each tree was trained on a bootstrapped sample of the training data and a random subset of features, and final predictions were made by aggregating probability estimates across all trees. The number of estimators and maximum tree depth were tuned to balance performance against computational cost. Class imbalance was addressed using balanced class weighting.

4.1.3 LOF Implementation

The LOF model was implemented using scikit- learn’s LocalOutlierFactor class with novelty detection mode enabled.

Implementation Details:

- Training samples: 50,000 normal transactions
- Features: 320
- k (neighbors): 20
- Distance metric: Manhattan
- Contamination: 0.0343 (validation fraud rate)
- Training time: ~ 164 seconds

Threshold Selection:

Validation LOF scores were used to select the optimal threshold that maximizes the F1 score. Transactions with scores below this threshold are flagged as fraudulent.

4.1.4 KDE Implementation

The KDE model was implemented as a supervised, class- conditional density estimator. Two Gaussian KDEs were fitted separately, one on the legitimate transactions and one on the fraud transactions, using the same preprocessed and standard- scaled features. For each transaction, a fraud score was computed as the log- posterior. The bandwidth was tuned per class via grid search over $\{0.2, 0.4, 0.6\}$, evaluated on the validation set using the precision- recall curve. The final decision threshold was also selected on validation.

4.1.5 CNN Implementation

The CNN was implemented in Python using TensorFlow/Keras inside a Jupyter Notebook environment. The supporting libraries used for preprocessing and evaluation were NumPy, Pandas, Scikit-learn, and Matplotlib. The data was loaded from pre-split CSV files containing training, validation, and testing features and labels.

The neural-network input contained 209 preprocessed features. For the CNN, the input matrix was reshaped into $(\text{samples}, 209, 1)$ so that it could be processed by Conv1D layers. The implemented CNN architecture was:

1. Input shape: $(209, 1)$.
2. Conv1D layer with 32 filters and kernel size 2, followed by Batch Normalization and Dropout(0.2).
3. Conv1D layer with 64 filters and kernel size 2, followed by Batch Normalization and Dropout(0.5).
4. Flatten layer.
5. Dense layer with 64 neurons and ReLU activation, followed by Dropout(0.5).
6. Dense layers with 100, 50, and 25 neurons using ReLU activation.
7. Output layer with one sigmoid neuron for binary fraud classification.

The model was compiled using the Adam optimizer and binary cross-entropy loss. Accuracy and AUC were monitored during training. To address the severe class imbalance, class weights were applied with a higher weight assigned to the fraud class. Training was limited to 10 epochs with early stopping on validation AUC and restoration of the best weights.

4.1.6 ANN Implementation

The ANN was implemented using the same preprocessed input features but without reshaping the data into a convolutional sequence. Each transaction was passed directly as a 209-dimensional vector into a fully connected neural network.

The implemented ANN architecture was:

1. Input shape: (209) .
2. Dense layer with 256 neurons and ReLU activation, followed by Batch Normalization and Dropout(0.3).

3. Dense layer with 128 neurons and ReLU activation, followed by Batch Normalization and Dropout(0.3).
4. Dense layer with 64 neurons and ReLU activation, followed by Dropout(0.3).
5. Output layer with one sigmoid neuron for binary fraud classification.

The ANN was trained using Adam and binary cross-entropy loss. As with the CNN, class weights were used to reduce majority-class bias, and early stopping monitored validation AUC. This ensured that the ANN experiment followed the same evaluation conditions as the CNN while providing a faster dense-network baseline.

4.1.7 Reinforcement Learning Agent Implementation

The decision policy was implemented as a Proximal Policy Optimization (PPO) agent in PyTorch. The policy is parameterised by four learned scalars— $base_{low}$, $slope_{low}$, $base_{high}$, and $slope_{high}$ —which define the two decision thresholds as linear functions of the normalised log-amount $z(A)$. The policy and value functions are small multilayer perceptrons; the raw network outputs are passed through a bounded squashing transform so that thresholds remain within valid ranges, and the slope magnitude is capped at 0.20 to prevent degenerate policies.

Training used a batched sequential environment in which the agent observes the distribution of risk scores across a transaction batch and receives the realised financial profit as its reward. The hyperparameters were: discount factor $\gamma = 0.99$, learning rate 2×10^{-4} , entropy coefficient 0.02, PPO clip range 0.15, minibatch size 128, trained for 4000 episodes.

To prevent information leakage, the amount-normalisation statistics ($Z_{\text{LOGA_MEAN}}$ and $Z_{\text{LOGA_STD}}$) were computed on the validation set and frozen; the test set was never used during training, hyperparameter selection, or blend-weight selection, and was reserved exclusively for final evaluation. After training, the checkpoint achieving the highest validation profit was frozen and used for all subsequent evaluation and deployment; the agent is not retrained during operation. When the review budget or staffing level changes, only the lightweight capacity layer (Section 3.5.4) is re-tuned, while the PPO policy itself remains fixed.

4.1.8 Dashboard Implementation

The dashboard was implemented as an interactive Python application using Streamlit for the interface and Plotly for visualizations. Pandas and NumPy

were used for loading transaction outputs, computing displayed statistics, and preparing the tables and charts. The interface was designed to match the final system pipeline: a hybrid AE+CatBoost risk score is produced for each transaction, the PPO policy assigns an action, and the capacity layer adjusts the deployed review workload.

The dashboard interface is organized into two main modules. The first module is **Bank Operations**, which is designed for the employee workflow. It includes three subpages: Overview, Review and Override, and Risk Insights. The second module is **Offline Evaluation**, which is used to present the project results using the labeled test data. It includes Model Performance, Financial Evaluation, and Policy Comparison.

Table 2: Dashboard module structure.

Main module	Subpage	Function
Bank Operations	Overview	Presents operational KPIs, action mix, capacity usage, transaction risk distribution, and daily review load.
Bank Operations	Review and Override	Displays the review queue, selected transaction details, analyst decision controls, override controls, and audit trail.
Bank Operations	Risk Insights	Shows threshold behavior by transaction amount and card-group level risk patterns.
Offline Evaluation	Model Performance	Shows the evaluation summary and confusion matrix using the labeled test set.
Offline Evaluation	Financial Evaluation	Shows financial outcome comparison, cost breakdown, and detected versus missed fraud amount.
Offline Evaluation	Policy Comparison	Compares the global threshold baseline, RL amount-conditioned policy, and deployed capacity-aware policy.

Bank Operations Overview The Overview page displays the operational summary of the current transaction batch. It includes cards for transactions scanned, blocked transactions, transactions sent to review, blocked amount, total transaction value, average risk score, daily review capacity, and capacity utilization. These cards give the analyst a quick summary of the deployed system output and the current review workload.

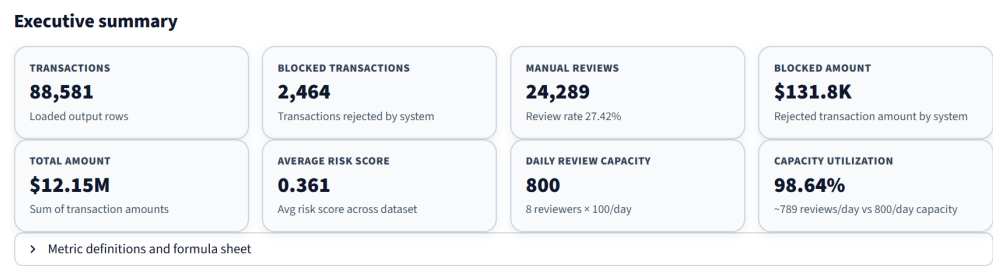


Figure 4: Bank Operations overview showing transaction count, blocked transactions, review workload, blocked amount, total transaction value, risk score, and review capacity indicators.

The same page also includes charts for the accept, review, and reject decision mix, as well as the used review capacity compared with the selected daily capacity. These charts show whether the system is sending a manageable number of transactions to manual review.

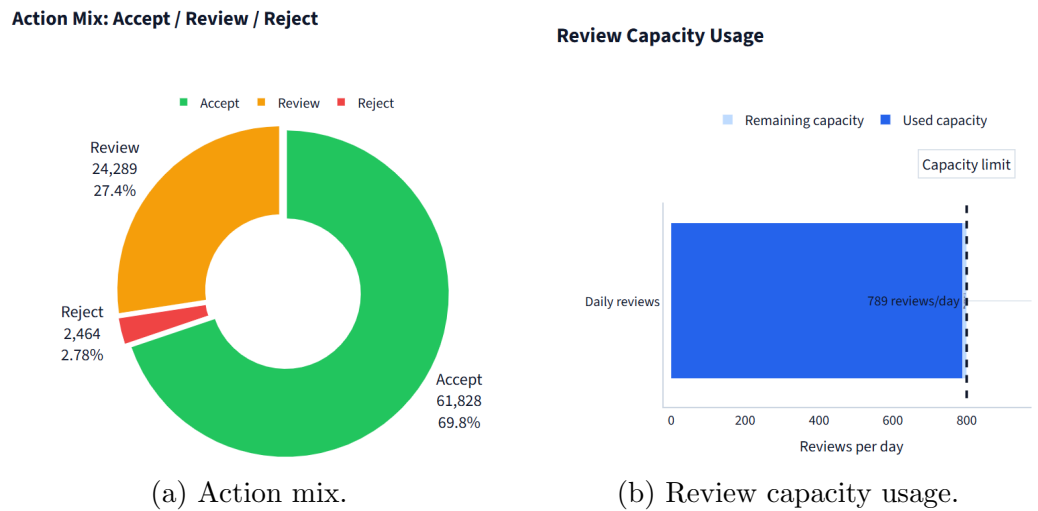


Figure 5: Operational decision analytics showing the action distribution and review capacity usage.

The Overview page also shows the risk-score distribution and the daily

review load. The risk distribution separates accepted, reviewed, and rejected transactions, while the daily review load compares the daily number of reviewed transactions against reviewer capacity. These visuals connect the model score distribution to the operational review burden.

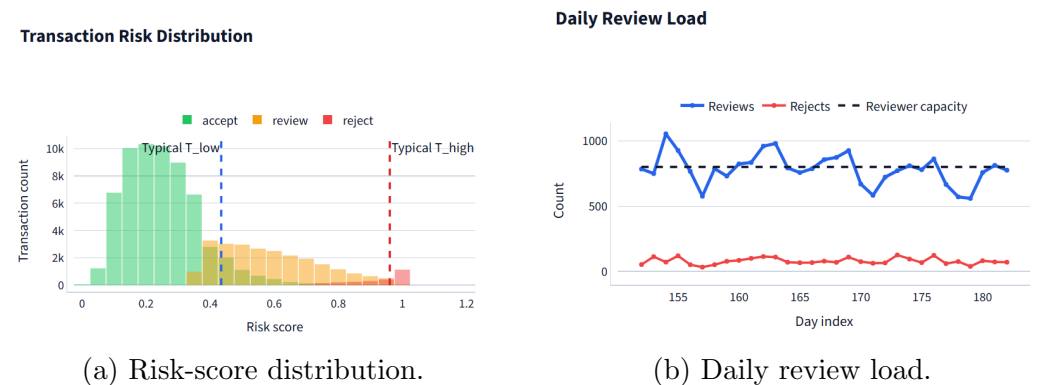


Figure 6: Operational workload visuals showing transaction risk distribution and daily review load.

Review and Override Workflow The Review and Override page contains the transaction review queue and the selected transaction details. The review queue displays transactions assigned to manual review and includes the risk level, transaction amount, card identifier, risk score, effective thresholds, RL action, and human review status. This provides the analyst with the information required to select and inspect suspicious transactions.

Prioritized review queue

Showing first 200 rows out of 24,289. Use export for the full file.

Risk level	TransactionID	TransactionAmt	card1	catboost_risk_score	effective_T_low	effective_T_high	rl_action	human_review_deci
Medium risk	3498770	117.0	15333	0.9793158653931505	0.36	0.98	review	not_reviewed_yet
Medium risk	3552824	212.106	10876	0.9793029762026862	0.36	0.98	review	not_reviewed_yet
Medium risk	3505101	350.0	9175	0.979118023270953	0.36	0.98	review	not_reviewed_yet
Medium risk	3494377	275.0	3111	0.9790794086732558	0.36	0.98	review	not_reviewed_yet
Medium risk	3519006	79.924	9026	0.978771272359186	0.40578260477008393	0.98	review	not_reviewed_yet
Medium risk	3489071	150.0	15066	0.9787517574143677	0.36	0.98	review	not_reviewed_yet
Medium risk	3505184	82.629	15944	0.9787411326286372	0.3995529665109214	0.98	review	not_reviewed_yet
Medium risk	3489070	150.0	8568	0.9784286476257543	0.36	0.98	review	not_reviewed_yet

Figure 7: Prioritized review queue showing transactions routed to manual review with their risk level, amount, risk score, thresholds, and RL action.

After selecting a transaction, the analyst can inspect the transaction details and choose a final human reviewer decision. The available actions are approve, reject, or hold for investigation. This keeps the analyst in control of the final decision for reviewed transactions while still using the model and RL policy to prioritize the workload.

Take action on transaction

Select Transaction ID

3573183

Human reviewer decision

☒ Approve

☐ Reject

☐ Hold for investigation

Notes

Optional review notes

Submit decision

Selected transaction details

Field	Value
TransactionID	3573183
TransactionAmt	433.0
card1	2616
catboost_risk_score	0.979962
effective_T_low	0.36
effective_T_high	0.98
Risk level	Medium risk

Figure 8: Transaction action panel showing selected transaction details and the available analyst decisions.

The same workflow includes a decision override form. This allows an analyst to search for a transaction, select the updated decision, choose a reason, and add notes. The action is then added to the audit trail. This feature is important for cases where additional information changes the appropriate action for a transaction.

Transaction ID

Enter TransactionID

Override system decision

Approve transaction

Reason

Customer confirmed legitimate

Override notes

Optional notes

Apply override

> Audit trail

Transaction selected for override

Enter a transaction ID to review and override its current system decision.

Figure 9: Decision override form for updating a transaction decision and recording the reason in the audit trail.

Risk Insights The Risk Insights page gives additional explanation of the deployed decision behavior. The high-risk transaction table highlights transactions with the highest scores and final blocked status. This helps analysts focus on the transactions that require the most attention.

High-risk transactions to monitor

Risk level	TransactionID	TransactionAmt	card1	catboost_risk_score	effective_T_low	effective_T_high	rl_action	final_status
High risk	3499636	375.0	1724	0.99997843070333	0.36	0.98	reject	blocked
High risk	3542870	22.993	9633	0.9998824961009835	0.6361260845611636	0.7359570547505749	reject	blocked
High risk	3569699	130.645	17147	0.999876169941243	0.36	0.98	reject	blocked
High risk	3542865	22.993	9633	0.9998691394376907	0.6361260845611636	0.7359570547505749	reject	blocked
High risk	3542862	22.993	9633	0.9998690767791373	0.6361260845611636	0.7359570547505749	reject	blocked
High risk	3542876	22.993	9633	0.9998644199470547	0.6361260845611636	0.7359570547505749	reject	blocked

Figure 10: High-risk transactions table showing transactions with high risk scores, thresholds, RL action, and final status.

The page also shows the RL threshold behavior by transaction amount and the card-group amount and risk graph. The threshold plot shows how the effective lower and upper thresholds behave across transaction amounts, while the card-group plot summarizes card-related groups using total transaction amount and average risk score. Together, these visuals explain how transaction amount, risk score, and grouped transaction patterns affect the final decision.

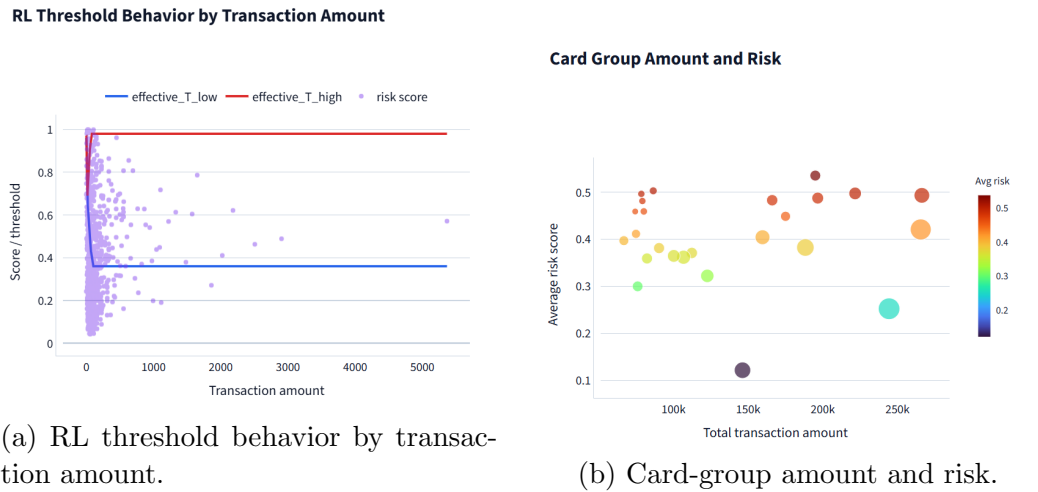


Figure 11: Risk Insights visuals showing threshold behavior and card-group risk patterns.

Dashboard Data Files The dashboard uses exported outputs from the completed system pipeline. The transaction-level file contains transaction amount, risk score, decision thresholds, RL action, final status, and cost fields. The metrics file contains the aggregate results such as action counts, review rate, precision, recall, total cost, protected exposure, and profit gain. The policy file contains the reviewer settings, review capacity, threshold parameters, and deployment settings. The policy comparison file contains the comparison between the global threshold baseline, the RL amount-conditioned policy, and the deployed capacity-aware policy.

4.2 Results

4.2.1 Autoencoder and Hybrid Classifier Results

The performance of the fraud detection system was evaluated on a held-out test set containing 88,581 transactions, of which 3,083 were actual frauds. The system’s performance was measured across two distinct architectural phases: the unsupervised Autoencoder Ensemble baseline and the Hybrid Autoencoder + CatBoost classifier.

Phase 1: Autoencoder Ensemble Baseline Results For the Autoencoder Ensemble, the decision threshold was calibrated on the validation set to target a False Positive Rate (FPR) of under 0.12 before predicting on the test set. The purely unsupervised ensemble model yielded the following quantitative metrics on the test set:

- Overall Accuracy: 84%
- Precision-Recall AUC (PR AUC): 0.1211
- Receiver Operating Characteristic AUC (ROC AUC): 0.7689
- Recall (Sensitivity): 0.5537 (Detecting roughly 55.3% of actual fraud)
- Precision: 0.1206
- F1-Score: 0.1980
- False Positive Rate (FPR): 0.1457

Phase 2: Hybrid Model Results (Autoencoder + CatBoost) For the hybrid classifier, the decision threshold was instead selected by maximising the F1-score on the validation set—consistent with the protocol used for LOF—and applied unchanged to the test set. The hybrid framework, which blends the CatBoost probability (weighted at $\alpha = 0.95$, chosen by validation profit) with the Autoencoder’s anomaly percentile, yielded the following test metrics:

- Overall Accuracy: 96.8%
- Precision-Recall AUC (PR AUC): 0.5292
- Receiver Operating Characteristic AUC (ROC AUC): 0.9127
- Recall (Sensitivity): 0.5105 (Detecting roughly 51.1% of actual fraud at the fixed threshold)
- Precision: 0.5370
- F1-Score: 0.5234
- False Positive Rate (FPR): 0.0159

4.2.2 Random Forest Results

On the held- out test set, the Random Forest model achieved an AUC- ROC of approximately 0.90, reflecting strong ability to rank fraudulent transactions above legitimate ones. Precision and recall were both approximately 0.51, yielding an F1- score of 0.51. While this means roughly half of flagged transactions are true fraud and half of all fraud cases are caught. This balance, together with the high AUC-ROC, confirms that the model’s underlying risk scoring is reliable. Random Forest therefore served as a strong supervised baseline against which the chosen hybrid classifier was compared.

4.2.3 LOF Results

The grid search selected a threshold of -2.126. On the held- out test set, the LOF model achieved a ROC AUC of approximately 0.719, with a recall of about 35% and a precision of about 12% at the chosen threshold.

- Fraud caught: 1,076 out of 3,083 (34.9%)
- False alarm rate: 9.0%
- F1 score: 0.18

4.2.4 KDE Results

The grid search selected a bandwidth of 0.4 for the legit KDE and 0.6 for the fraud KDE. On the held- out test set, the KDE model achieved a ROC AUC of approximately 0.70, with a recall of about 41% and a precision of about 14% at the chosen threshold. These numbers show that KDE is able to pick up a meaningful share of fraudulent transactions while keeping the model lightweight and interpretable, but it falls short of the supervised models in raw detection performance. The relatively low precision is expected, since density- based scoring tends to flag any transaction that lies in a low- density region of the legit class, even when it is just an unusual but genuine purchase. This makes KDE a useful component in the wider pipeline rather than a standalone solution, particularly as a fast, distribution- free signal that can be combined with the stronger supervised models.

4.2.5 CNN and ANN Results

The CNN and ANN were evaluated on the held-out test set using accuracy, ROC-AUC, fraud-class precision, fraud-class recall, fraud-class F1-score, and approximate training time per epoch. The Basic CNN experiment from the notebook was not included in the thesis comparison; only the final CNN and ANN models were considered.

Table 3: CNN and ANN test-set performance.

Metric	CNN	ANN
Accuracy	95.09%	94.27%
ROC-AUC	0.8796	0.8874
Fraud precision	0.37	0.32
Fraud recall	0.54	0.58
Fraud F1-score	0.44	0.41
Training time per epoch	~ 220 s	~ 7 s

The CNN achieved the highest accuracy and the strongest fraud F1-score among the two neural-network baselines. Its fraud precision of 0.37 means that it produced fewer false fraud alarms than the ANN, while still detecting more than half of the fraud cases in the test set. The ANN achieved a slightly higher ROC-AUC of 0.8874 and a slightly higher fraud recall of 0.58, meaning it ranked fraudulent transactions well and caught a larger proportion of fraud cases. However, its fraud precision was lower, which reduced its fraud F1-score.

The difference in training cost was significant. The ANN required approximately seven seconds per epoch, while the CNN required around 220 seconds per epoch. This is expected because the ANN only uses dense layers, while the CNN applies convolutional operations before classification. Therefore, the CNN provided stronger fraud-class balance, while the ANN served as a much faster deep-learning comparison model.

Table 4: Summary of model results on the held-out test set.

Model	AUC	Precision	Recall	F1-score
Autoencoder ensemble	0.7689	0.121	0.554	0.198
Hybrid AE+CatBoost	0.9127	0.529	0.518	0.523
Random Forest	~ 0.90	~ 0.51	~ 0.51	~ 0.51
LOF	0.719	0.12	0.35	0.18
KDE	~ 0.70	0.14	0.41	–
CNN	0.8796	0.37	0.54	0.44
ANN	0.8874	0.32	0.58	0.41

The summary table places the neural-network baselines beside the other tested models. The hybrid AE+CatBoost model remained the strongest risk scorer for the final RL pipeline, while the CNN and ANN provided additional deep-learning baselines for comparison.

4.2.6 Reinforcement Learning Policy Results

The amount-conditioned PPO policy was evaluated on the held-out test set of 88,581 transactions. The deployed policy achieved a test profit of \$67,530 at a review rate of 26.9%, corresponding to a Profit Gain of 0.882—that is, it captures 88.2% of the value achievable between the do-nothing baseline (0%) and the perfect-oracle ceiling (100%). Averaged across five independent random seeds, the policy achieved a **test** profit of \$67,838 $\pm$ \$490. The convergence plot (Figure X) shows the corresponding **validation** profit, which converges to an even tighter band ($\pm$ \$22); the wider test spread is expected, since the policy is selected on validation and evaluated on unseen test data.

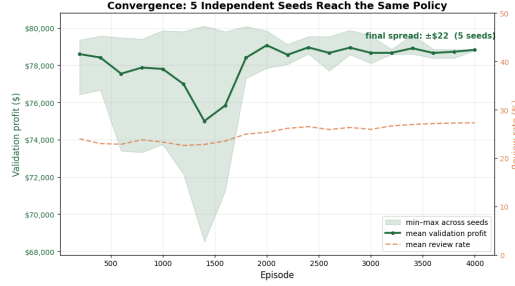


Figure 12: Validation profit across training episodes for five independent seeds. The seeds converge to a tight band (spread of approximately \$22 on validation), confirming stable, reproducible training.

Value of continuous amount-conditioning To confirm that the *learned, continuous* amount-conditioning is responsible for the performance—rather than amount-conditioning in any form—the policy was compared against an amount-bucketed grid search, which discretises transactions into four log-amount quartiles and tunes a fixed threshold pair within each. On the same test set, the bucketed grid achieved \$60,257 (Profit Gain 0.877) at a 24.3% review rate, while the continuous PPO policy achieved \$67,530—an advantage of \$7,272.

This gap is statistically significant, not an artifact of a single evaluation. A moving-block bootstrap over contiguous time windows (block length 2,214 transactions, 41 blocks, 4000 resamples) places the 95% confidence interval of the advantage at [\$1,589, \$15,541], with an estimated probability that the true gap is non-positive of only $P \approx 0.003$. The continuous learned policy therefore reliably outperforms hand-partitioned amount bucketing, confirming that the smooth, learned conditioning—not merely the presence of amount-awareness—is the source of the improvement.

Necessity of the autoencoder component A natural question is whether the autoencoder earns its place, given that CatBoost carries the dominant blend weight ($\alpha = 0.95$). To test this, the entire pipeline was re-run with the autoencoder removed, using the CatBoost score alone as the risk input to the same PPO policy, across five seeds. The CatBoost-only configuration achieved $\$56,558 \pm \$1,912$, whereas the full hybrid achieved $\$67,838 \pm \490 —an advantage of roughly \$11,000 in realised profit together with a nearly four-fold reduction in seed-to-seed variance. The small $(1 - \alpha)$ autoencoder contribution therefore has an effect on downstream profit far out of proportion to its blend weight: it stabilises the score distribution the policy depends on and materially improves the operating point. This result is the empirical

basis for retaining the autoencoder rather than reverting to a CatBoost-only scorer.

Negative control As a check against inadvertent label leakage, the deployed system was re-evaluated on the test set with the fraud/legitimate labels randomly shuffled while scores, amounts, and the policy were held fixed. Test profit collapsed from +\$67,890 on the real labels to −\$449,468 under shuffling, confirming that the policy’s performance depends on genuine fraud structure and is not an artefact of the data ordering or split.

4.2.7 Capacity Layer Results

The capacity layer was evaluated on the test set using the deployed mechanism: a budget re-tuned static policy combined with pressure-based triage, applied to the frozen PPO policy. Because the review budget is an operational parameter, results are reported as a function of analyst staffing rather than as a single number.

- At a budget of 20.85% (six analysts), the system achieved a test profit of approximately \$46,524 with a realized review rate of 20.85%.
- At a budget of 27.8% (eight analysts), the system achieved a test profit of approximately \$67,890 with a realized review rate of 27.4%, corresponding to a Profit Gain of 88.2% relative to the unconstrained profit ceiling.

An ablation confirmed that the triage stage contributes genuine value beyond simple amount-gating: pressure-gated triage outperformed both a flat budget-constrained policy and an amount-only gating variant, indicating that adapting to real-time budget pressure—not merely transaction size—is responsible for the gain. Because staffing is a post-training dial, moving between the six- and eight-analyst operating points requires no retraining, only re-derivation of the lightweight static offsets.

4.2.8 Dashboard Output and Visual Analysis

The dashboard was used to present the final output of the fraud detection system in both operational and evaluation forms. In the Bank Operations view, the dashboard summarizes the deployed decisions using operational indicators such as total transactions, accepted transactions, transactions sent to review, rejected transactions, blocked amount, total transaction value, average risk score, daily review capacity, and capacity utilization. These

values describe how the model output is converted into an analyst-facing workflow.

The operational charts show the distribution of system actions across accept, review, and reject decisions. The review capacity chart compares the daily review workload against the available reviewer capacity, while the daily review load chart shows whether the number of reviewed transactions remains manageable under the selected staffing level. The transaction risk distribution shows how the risk scores are spread across the batch and how the accept, review, and reject regions are separated.

The Review and Override page demonstrates the analyst workflow. Transactions assigned to review can be selected from the queue, inspected individually, and given a final analyst action. The available actions are approve, reject, or hold for investigation. If the analyst changes the system decision, the dashboard records the decision in the audit trail together with the original system action, final analyst action, reason, and timestamp. This makes the dashboard more than a static visualization page, because it supports direct interaction with the decision process.

The Offline Evaluation view presents the project results using the labeled test data. The Model Performance page shows the evaluation summary and confusion matrix. The evaluation summary reports the AUC-ROC, precision, recall, total cost, and caught fraud value. The confusion matrix shows how many transactions were classified as fraud or legitimate according to the test labels and the deployed decision logic.

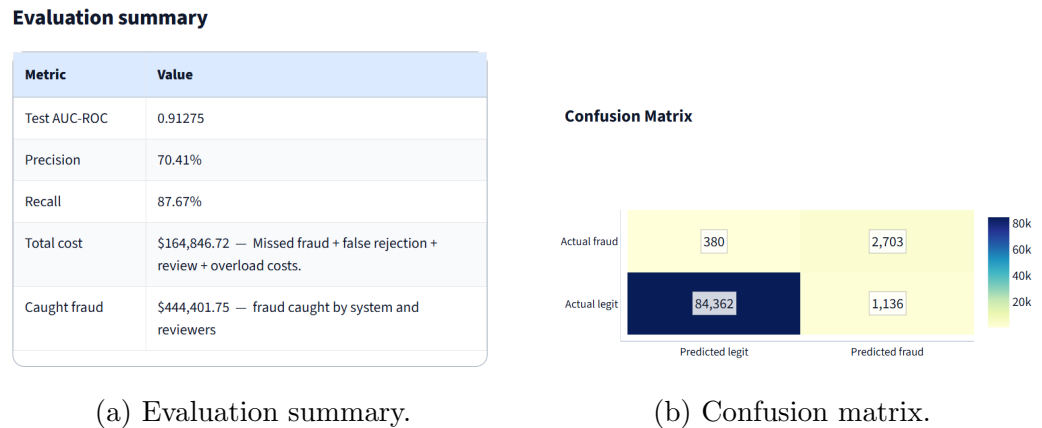


Figure 13: Offline Evaluation model performance view showing summary metrics and confusion matrix results.

The Financial Evaluation page presents the business effect of the deployed system. The financial outcome comparison shows the difference between no

fraud management, the proposed Hybrid AE+CatBoost+RL system, and the oracle reference. The detected versus missed fraud amount chart shows the fraud amount caught by the system and reviewers compared with the amount missed. The cost breakdown separates the total cost into missed fraud, false rejection, manual review, and overload costs.

Financial Outcome Comparison

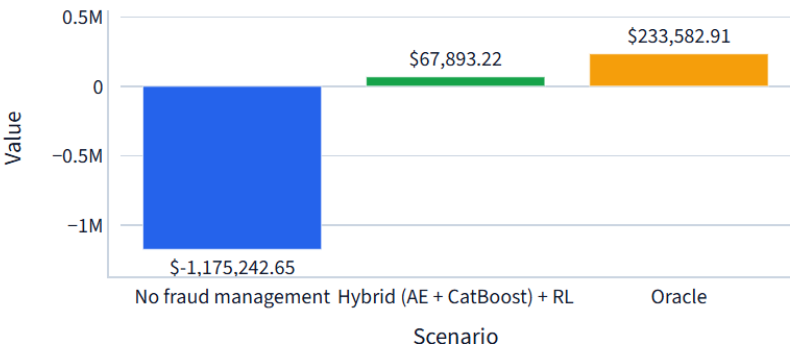
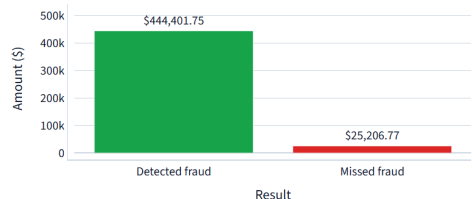


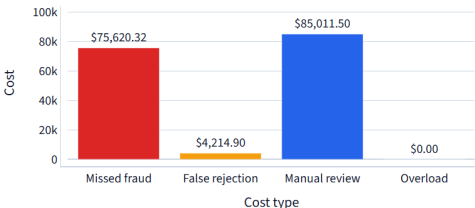
Figure 14: Financial outcome comparison between no fraud management, the proposed Hybrid AE+CatBoost+RL system, and the oracle reference.

Detected Fraud vs Missed Fraud Amount



(a) Detected versus missed fraud amount.

Cost Breakdown



(b) Cost breakdown.

Figure 15: Financial Evaluation visuals showing caught fraud amount, missed fraud amount, and cost components.

The Policy Comparison page compares the global threshold baseline, the RL amount-conditioned policy, and the deployed capacity-aware policy. The

total-cost chart shows that the RL-based approaches reduce cost compared with the global threshold baseline. The profit-gain chart shows the resulting improvement in the business objective. The threshold comparison visualizes the lower and upper decision thresholds used by each policy.

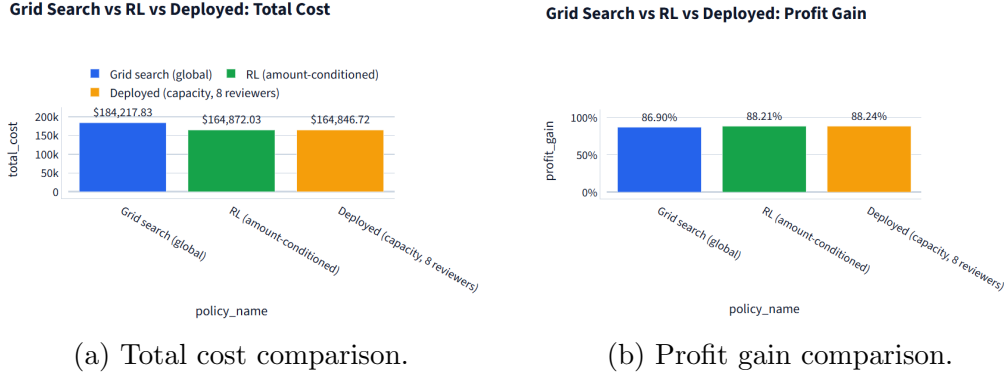


Figure 16: Policy comparison between global thresholding, RL amount-conditioned policy, and deployed capacity-aware policy.

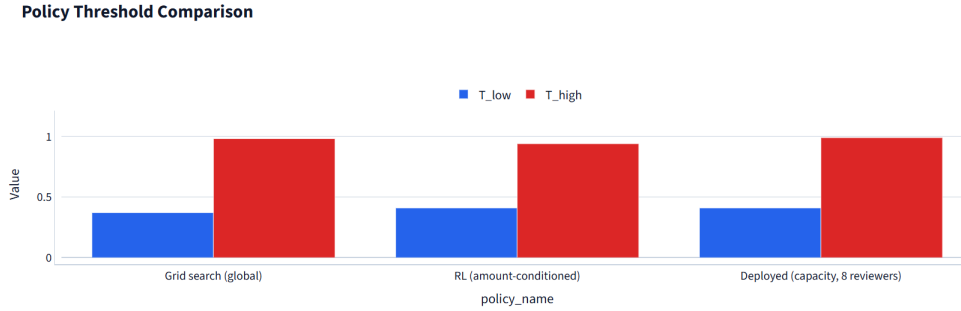


Figure 17: Policy threshold comparison showing the lower and upper decision thresholds for the evaluated policies.

Overall, the dashboard output shows both sides of the system. The Bank Operations view demonstrates how the system supports financial experts during transaction monitoring, while the Offline Evaluation view presents the measured project results using the labeled dataset. This allows the same dashboard to support both the analyst workflow and the academic evaluation of the proposed ML and RL approach.

4.3 Analysis of Results

4.3.1 Autoencoder Analysis

The Autoencoder Ensemble achieved a recall of 55.4% and a low precision of 12.1%. This reflects a key limitation of purely unsupervised anomaly detection: while it successfully flags unusual transactions, it struggles to differentiate between highly irregular legitimate spending and actual fraud, resulting in a 14.6% false alarm rate. With a PR AUC of 0.121 and an ROC AUC of 0.769, the ensemble serves best as a robust feature extractor rather than a standalone operational classifier.

This precision ceiling is intrinsic to label-free scoring: the autoencoder can only measure how unusual a transaction is, not whether that unusualness corresponds to fraud. Pairing it with a supervised CatBoost component—which learns the actual fraud/legitimate boundary from labels—directly targets that gap, adding discriminative power the ensemble structurally cannot provide while retaining its sensitivity to novel patterns. The resulting hybrid classifier (Phase 2, Section 4.2.1) improves both ranking ability and precision over the ensemble, which is why it was adopted as the risk scorer for the reinforcement-learning layer.

4.3.2 LOF Analysis

The LOF model achieved 34.9% recall with a 9.0% false alarm rate. Its low precision (12%) reflects a key limitation of unsupervised methods: without labeled fraud examples during training, the model cannot distinguish between unusual legitimate transactions and actual fraud. The ROC AUC of 0.719 indicates moderate discriminative ability, making LOF suitable as a baseline or feature source rather than a standalone classifier.

4.3.3 KDE Analysis

As noted, KDE achieves a recall of 41% but precision of only 14%, reflecting the inherent limitation of density-based methods: they flag any transaction in a low-density region of the legitimate class, including unusual but genuine purchases. Nevertheless, KDE’s simplicity, interpretability, and distribution-free nature make it a valuable component in an ensemble or as a fast baseline.

4.3.4 Random Forest Analysis

With AUC-ROC of 0.90 and precision/recall both ~ 0.51 , Random Forest provides a balanced and reliable risk score. The F1-score of 0.51 indicates

that while half of flagged transactions are true fraud and half of frauds are detected, the model’s probability estimates are well-calibrated. Random Forest thus stands as the strongest single supervised baseline, though the hybrid AE+CatBoost classifier was ultimately selected as the risk scorer for the RL layer.

4.3.5 CNN and ANN Analysis

The CNN and ANN results show the tradeoff between stronger feature extraction and computational cost. The CNN achieved a fraud F1-score of 0.44, outperforming the ANN’s fraud F1-score of 0.41. This indicates that the convolutional layers helped the model extract useful local feature interactions before the dense classification stage. The CNN also achieved higher fraud precision, meaning it produced fewer false fraud alarms among the transactions it flagged.

The ANN, however, achieved the higher ROC-AUC and trained much faster. Its AUC of 0.8874 shows that the dense network ranked fraudulent transactions slightly better overall, while its training time of approximately seven seconds per epoch made it much lighter computationally. Its lower precision means that the improvement in recall came with more false alarms. Overall, the CNN was the stronger neural-network baseline in terms of fraud-class F1-score, while the ANN was useful as a fast dense comparison model.

5 Chapter 5: Discussion

5.1 Interpretation of Results

Across the supervised baselines, Random Forest and the hybrid AE+CatBoost classifier performed comparably in raw ranking ability (test AUC-ROC 0.90 and 0.9127 respectively), both clearly ahead of the density- and distance-based methods (KDE, LOF) and the standalone autoencoder ensemble. The hybrid AE+CatBoost was selected as the risk scorer for the reinforcement-learning layer not primarily for this modest AUC margin, but because its score distribution—deliberately left uncalibrated and reshaped to spread across the decision range—is what the amount-conditioned threshold policy requires in order to operate, and because its blend weight was chosen to maximize validation profit rather than accuracy. This reflects the central premise of the thesis: the classifier is judged by how well it enables profitable downstream decisions, not by its standalone accuracy.

5.2 Comparison with Previous Studies

Our models generally reported lower headline detection numbers than several of the studies surveyed in Chapter 2. We attribute this primarily to the difficulty of the IEEE-CIS dataset—its heavy anonymisation of features, extreme class imbalance, and pronounced temporal drift—which depressed performance uniformly across every model we evaluated, supervised and unsupervised alike. More importantly, a direct comparison of raw detection scores is not the appropriate yardstick for this work: the surveyed papers optimize for classification quality, whereas our system is optimized for realised financial profit under an explicit cost model. A classifier with a lower standalone F1 can still yield a superior operating point once its scores are routed through the cost-aware decision policy, which is the quantity our framework is designed to maximise.

5.3 Limitation

The Challenges when doing this study were:

- Long Training Time which exceeded 12 hours on multiple occasions, this was solved by setting our device to its highest setting and using external cooling
- Temporal Drift which collapsed performance for the model which was solved through feature engineering

6 Chapter 6: Conclusion and Future Work

6.1 Summary of Work

To summarize, our goal was to minimize cost for financial institutions and detect financial fraud cases, which can be shown intuitively by the strong recall of the hybrid classifier feeding the RL agent. We achieved this using a hybrid AE+CatBoost classifier that reached a test AUC-ROC of 0.9127, selected specifically for its suitability as the input to the reinforcement-learning cost-optimization layer.

6.2 Key Contributions

The key innovations we achieved were the design of the hybrid AE+CatBoost classifier and the amount-conditioned reinforcement-learning policy layered

on top of it. As for the cost function, we improved on the foundations previously laid out by other papers, combining them into a more comprehensive cost function that evaluates the model and also assesses its performance from a business perspective.

6.3 Future Work

To further improve our proposed solution, the following additions are needed:

- Learn the cost parameters from institutional data instead of assuming parameters. As this ensures that a more optimal solution.
- Online adaptation to concept drift For a more dynamic policy to eliminate the need to do full re-training for the model.
- Integrate into HDFS for Faster training and processing time.
- Real-time streaming deployment into the analyst dashboard.

References

- [1] Statista, "Total value of losses due to card fraud worldwide from 2014 to 2023, with forecasts to 2028," 2024.
- [2] Statista, "Number of payment cards in circulation worldwide," 2024.
- [3] Statista, "Global credit card transaction volume and payment trends," 2024.
- [4] A. C. Bahnsen, A. Stojanovic, D. Aouada, and B. Ottersten, "Cost Sensitive Credit Card Fraud Detection Using Bayes Minimum Risk," in Proc. IEEE 12th Int. Conf. Machine Learning and Applications (ICMLA), 2013, pp. 333-338, doi: 10.1109/ICMLA.2013.68.
- [5] S. Hoppner, B. Baesens, W. Verbeke, and T. Verdonck, "Instance- dependent cost- sensitive learning for detecting fraud," Expert Systems with Applications, 2021.
- [6] M. Y. Yildirim, M. Özer, and H. Davulcu, "Cost- Sensitive Decision Making for Online Fraud Management," in Proc. IFIP Int. Conf. Artificial Intelligence Applications and Innovations (IAIAI), 2018, pp. 323-336, doi: 10.1007/978-3-319-92007-8_28.

- [7] "Credit Card Fraud Detection Using Autoencoder Neural Network"
<https://arxiv.org/pdf/1908.11553>
- [8] P. Ram and A. G. Gray, "Fraud Detection with Density Estimation Trees," in Proc. KDD Workshop on Anomaly Detection in Finance, vol. 71, pp. 85-94, 2017.
- [9] M. M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sander, "LOF: identifying density- based local outliers," in Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, Dallas, TX, USA, 2000, pp. 93–104.
- [10] S. P. Maniraj, A. Saini, S. D. Sarkar, and S. Ahmed, "Credit card fraud detection using machine learning and data science," International Journal of Engineering Research & Technology (IJERT), vol. 8, no. 9, pp. 110–115, Sep. 2019.
- [11] S. Sugidamayatno and D. Lelono, "Outlier detection credit card transactions using Local Outlier Factor algorithm (LOF)," IJCCS (Indonesian Journal of Computing and Cybernetics Systems), vol. 13, no. 3, pp. 233–242, 2019.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [13] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 3146–3154.
- [14] Stripe, "How Stripe uses machine learning to fight fraud," Stripe Radar Documentation, 2024.
- [15] T. Verbraken, W. Verbeke, and B. Baesens, "A novel profit maximizing metric for measuring classification performance of customer churn prediction models," IEEE Transactions on Knowledge and Data Engineering, vol. 25, no. 5, pp. 961–973, 2013.
- [16] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," Expert Systems with Applications, vol. 41, no. 10, pp. 4915–4928, 2015.

- [17] X. Liu, J. Li, and P. S. Yu, "Adaptive fraud detection with deep reinforcement learning," in Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM), 2021, pp. 1045–1054.
- [18] B. Baesens, R. J. Bolton, and T. Verdonck, "Benchmarking state-of-the-art classification algorithms for credit card fraud detection," *European Journal of Operational Research*, vol. 290, no. 2, pp. 557–571, 2021.
- [19] E. Esenogho, I. D. Mienye, T. G. Swart, and K. A. A. Akinlabi, "A deep reinforcement learning framework for credit card fraud detection," *IEEE Access*, vol. 10, pp. 58320–58334, 2022.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

Team Workload Assignment

Student Name	Responsibility	Tasks Accomplished	Additional Contribution
Mohamed Walid	RL Cost Function	Cost function established successfully.	Capacity layer implemented successfully.
Mohammad Ossama	Autoencoder + Data Processing	Data processing completed and autoencoder model trained successfully.	RL agent completed successfully and contributed to the capacity layer integration.
Ammar Ahmed	KDE	KDE model trained successfully.	Contributed to the RL model development.
Jana Sameh	CNN	CNN model trained successfully.	Dashboard integrated successfully.
Aly Osman	Random Forest (RF)	Random Forest model trained successfully.	Dashboard integrated successfully.
Abdulrahman Amr	Local Outlier Factor (LOF)	LOF model trained and validated successfully.	Dashboard integrated successfully.